8 A Software Product Line Reference Architecture for Security

T.E. Fægri and S. Hallsteinsen

Abstract.

Security is a cross-cutting concern in software intensive systems and should consequently be subject to careful architectural analysis and decision making. The requirements for cost-effective product line development complicate this task. Two central research questions are addressed in this chapter (1) Is it viable to represent architectural security knowledge in a reference architecture? (2) If so, is such a reference architecture useful for security architecture design in software product lines? Initial evidence suggests that both questions can be affirmed. The main contribution of this chapter is a reference architecture that draws upon state-of-the-art techniques and practices from software product line engineering and information security and constitutes a decision support framework for security architecture design in software product lines. To validate the reference architecture, the chapter also presents our experiences from using it at three distinct companies.

8.1 Introduction

Increasingly, security related requirements constitute a significant portion of the total set of requirements for many software systems. Arguably, the most important aspect contributing to this trend is the seemingly continually growing demand for more open and flexible IT systems. Terms such as "the real-time enterprise," "software infrastructures," "service oriented architectures" and "composite software applications" proliferate in the corporate IT arena and denote information systems that support cross-application integration, cross-company transactions and end-user access through a range of channels, including the Internet. For product oriented companies these trends are important too, because most applications will in some form interact with other applications. Although this is a natural consequence of the desire to improve the operational efficiency and reduce the need for manual work, application integration and Internet access make critical assets vulnerable to many threats. For most product oriented companies, requirements for security are likely to be as varied as for any other quality. Thus it can be expected that companies will want to supply variants of the same product to satisfy the variability in product requirements.

The architecture of a software system is important for the system's ability to satisfy its requirements [2–4, 8, 14, 25, 39]. In other words, if the architecture is carefully designed the resulting system has a better chance of meeting its expectations. Constructing software systems of any significant size or complexity requires considerations to the architecture.

By architecture we mean its conceptual organization in components, connectors and the relations between them. This is an abstract view of the software system that allows us to reason about high level aspects such as security, performance, maintainability, deployment, and functionality without having to consider all the details.

The art of creating architectures is normally performed by people with lots of experience in the particular domain of the application. Experience creates valuable knowledge. In an effort to manage this knowledge, the software architecture community has created the architectural pattern concept. Architectural patterns are working principles that have proven useful in architectural design and have been documented so that others can reuse the knowledge. We argue that through careful management of this knowledge, for example in terms of architectural patterns, software architectures can be created more effectively and with a higher probability of achieving the desired qualities. However, the existence of architectural patterns is not enough to facilitate the construction of good architectures. No set of patterns will create an architecture that is optimal for all stated requirements. We must also capture and reason upon the various effects of these patterns. After all, architectural design is all about making sound tradeoffs. Architectural patterns, accompanied with knowledge of their effects, help us in making good design decisions.

Making these tradeoffs effectively becomes even more important in a context where a company wants to deliver multiple product variants to the market. While seeking to minimize the global cost of producing those products, the variability in quality requirements will favor a systematic approach to architectural design where variation among member products can be precisely managed. We build upon the large volume of research and experience within the area of Software Product Lines (SPL). SPL is an approach to software development that seeks to optimize productivity by assisting strategic reuse of software assets [34]. SPL incorporates methodologies for capturing and planning for variations among a group of products.

Security is a quality aspect of software systems that must be addressed by the architecture. It is a cross-cutting concern that is affected by a wide range of architectural decisions. For example, a software system that is constructed using third party components needs to have an architecture that is able to contain potentially malicious components within security boundaries. Simultaneously, the software architect is normally forced to balance this concern against a number of other, potentially conflicting concerns. The reference architecture presented here supports the software architect in the SPL approach. We treat security requirements as a natural source of variability among the product members. In order to capture and manage knowledge related to security architectural design we propose a reference architecture for software product line engineering. It is in essence a knowledge repository with a structure to support architectural design. It consists of

1. a quality model representing and organizing our vocabulary for security requirements,
2. a decision model constituted by the scenarios that represent the security requirements for the application to be designed
3. a security architecture language prescribing architectural solutions to the security requirements.

The structure of the remaining part of this contribution is as follows: Section 8.2 discusses the construction of software architectures facing security requirements. Section 8.3

describes the main theoretical framework for the reference architecture through conceptual models. Sections 8.4–8.6 constitute the reference architecture documenting the quality model, the decision model and the security architecture language, respectively. Section 8.7 elaborates upon how to use the reference architecture and Sect. 8.8 describes our experiences with using the reference architecture in practice. Section 8.9 presents work that is related to ours. Section 8.10 presents concluding comments.

8.2 Security Architecture Design

Software architecture is concerned with the overall structure of software systems. We generally adhere to the IEEE 1471 definition: "The fundamental organization of a system embodied in its components, their relationships to each other, and to the environment, and the principles guiding its design and evolution." p.3 in [27]. Thus, architectural design deals with making high level decisions regarding the overall organization of the software system. As most successful systems can be expected to have a long lifespan, the efforts required to maintain the system are a key concern.

The architecture of a software system has a great impact on its ability to satisfy its requirements. Conversely, making changes to the architecture of an already existing system can be very expensive. Furthermore, ensuring that the software system is able to remain compliant with its quality requirements is just as important [6]. Thus, the architecture becomes a critical asset for the organization. Therefore the architecture should undergo a thorough design and evaluation process.

Most software products have many stakeholders with different roles who want to influence the business drivers and the quality requirements set for the final system. One challenge in this work is to specify quality requirements in a way that makes them clear and testable. The software architect must carefully search for architectural constructions that promise to address the requirements in the best possible way. While designing the architecture, tradeoffs should be made explicitly to support an open design process involving the relevant stakeholders. Architecture design is not a simple task, neither is it a task that lends itself easily to automation.

8.2.1 Encoding Architectural Knowledge

As mentioned in the introduction, architectural design is a knowledge intensive art that depends heavily upon experience. In order to encode and reuse this knowledge, the software architecture community has created the concepts architectural tactics and architectural patterns. They are all documented, reusable architectural solutions that promise to address specific concerns in software architectures. They are, essentially, representations of knowledge of how particular problems can be solved [51].

As the number of solutions increases, so does the need to see relationships between them. Therefore, it is useful to put these architectural solutions into a system. Architecture

solutions occur in widely different contexts, but they may nevertheless have a lot of similar characteristics. Also, architectural solutions occur at different levels of abstraction. Some are merely tactics in the solution space; others are very specific – prescribing components, interactions and roles. Generally we can say that architectural tactics are less specific than architecture patterns, but it is not possible to draw distinct borders between them. Architecture solutions form a continuum, where the level of abstraction is a viable dimension for considering them.

A reference architecture is a guideline for the design of architectures within a given domain, i.e., it is a recommendation for how to build a particular kind of system. One can say that a reference architecture is the architecture of a set of architectures. Typically, the main rationale for constructing reference architectures is the desire to capture, represent and share knowledge about what the requirements for certain types of systems are and how to build the systems, thus helping to standardize types of architectures. In the described reference architecture, we have systematized architectural solutions for security requirements. We call this system the security architecture language (see Sect. 8.6).

8.2.2 Security Design

Security design draws upon a large body of knowledge. Security has been a concern for computer system designers almost from the outset; the early systems were often used in sensitive military applications. Security then became a mainstream requirement with the advent of multi-user computers in the late 1960s when commercial use triggered concerns of malicious behavior from other users [17]. Since then, security has seen an increasing popularity. In the last years, as the attention to Internet based computing has exploded, security has again been a top priority in many fora.

Security deals with protecting assets and making sure that they remain valuable to their owner. In order to accomplish that, we must determine the relevant threats towards the assets, i.e., construct a risk assessment profile. Only after having gathered a good understanding of the threats facing the system can we make good decisions for what countermeasures we should introduce. The security submodel of the reference architecture (see Sect. 8.3) describes the conceptual model for how to create the risk assessment profile.

Security design introduces costs in terms of security technology, implementation and maintenance of security policies and effects on other quality attributes of the final software system (the latter aspect will be discussed in the next section). A key benefit of the risk assessment profile is that it provides support for deciding which investments in security should be made and which assets should be protected [24]. However, the task of determining *how* the assets should be protected must also be done.

It should be noted that this reference architecture only deals with the aspects of security that can be addressed through software. This might be called “logical security.” Other aspects of security, such as the design of physical countermeasures or barriers (this could be called “physical security”), are not covered by this reference architecture.

8.2.3 Security Architecture

As previously mentioned, architectural design involves making sound tradeoffs between design alternatives in order to achieve sufficient product quality [3, 4]. We base this work upon using architectural solutions (in the form of architectural tactics and patterns) as building blocks for software architectures, as previously described in [26], but also advocated by others, e.g., [2, 3, 39]. A principal idea is that each solution is associated with a set of effects on the quality attributes. By having an understanding of the effects from each architectural solution we can more easily reason about the total effects of all the architectural solutions used in the actual architecture under consideration. It should be noted that we do not aim at building a formally precise machine for determining the sum of effects from a set of architectural solutions. This is a hard problem due to the lack of precise metrics, the large amount of tacit knowledge going into architectural design and the complexity of dependencies between different decisions in the many layers of abstractions in a final system [13].

The security architecture language (Sect. 8.6) defines our solution space. These solutions are *countermeasures* that we can use in order to protect against damage to assets. These countermeasures represent knowledge about how to deal with various security threats, described by many previous efforts [5, 45, 48, 52, 54, 59].

Security architectural design is confronted with the same challenges as architectural design in general; certain tradeoffs must be made between important quality attributes. Usability, performance, etc. are frequently in conflict with security [23, 44, 53, 55, 57, 58, 61]. In situations like that, it is important to have a clear understanding of the tradeoffs which have been made [3, 14]. Through the systematic approach to architectural design presented here, we hope to make these tradeoffs more explicit. Subsequently, it will become easier to design architectures that maintain the interests of all stakeholders.

8.2.4 Security Architecture for Software Product Lines

For various reasons, it may be beneficial for an organization to deliver multiple products with overlapping capabilities to its markets. If overlapping capabilities are implemented in a controlled way, using the same assets, we call such a set of products a product line. Product lines bring the additional challenge of managing variability among similar products in a cost-efficient manner. The field of Software Product Lines (SPL) addresses these concerns, and has produced a large body of knowledge [7, 15, 34]. The presented reference architecture builds upon these ideas while applying them in a security-focused setting.

To reduce cost, an organization will typically strive to introduce a certain level of standardization of the architecture between the product line members [26]. This can be at different levels, for example in terms of technical platforms, prescribed frameworks, general quality requirements or recommended architectural solutions. In order to accommodate standardization, the product architect must first consider the implications of the already prescribed requirements and architectural solutions.

Already prescribed quality requirements must be reconciled with the product requirements. Architectural solutions identified as contributors towards the quality requirements of the product must be reviewed and aligned with already standardized solutions.

8.3 Conceptual Model of the Reference Architecture

This section presents a conceptual model (or view) of the reference architecture. The conceptual model illustrates the reference architecture at a high level of abstraction by showing how the central concepts relate to each other. Also, the concepts and their relationships are explained.

The term reference architecture was introduced in Sect. 8.2. For organizations building software product lines, reference architectures play an even more important role because they are an appropriate tool for the capture of standardized requirements and guidelines within the product line. The reference architecture *is* the product line architecture.

Inspired by the same rationale, we have built a reference architecture for security. It consists of three submodels:

1. A *security submodel* that supports the development of a risk assessment profile for the assets covered by the system. The risk assessment profile assists the software architect in deciding what requirements should be set for the system and their internal priorities. The risk assessment profile is also helpful in the process of determining the most appropriate countermeasures
2. An *architecture submodel* that incorporates architectural solutions which promise to address security related requirements
3. A *decision support submodel* that supports capturing, specifying and reasoning about requirements for the product line members. Requirements are formulated as scenarios representing variation points. One scenario represents one variation point. A variation point will normally represent multiple variants. Now, in the presented reference architecture, not all of the scenarios contain multiple variants. In the development of the guidelines, we decided it was useful to capture this security architecture design knowledge despite the lack of direct variability aspects.

Together, these three submodels give the software architect an integrated environment for architectural security design.

Figure 8.1 illustrates a conceptual model of the reference architecture. It shows the three submodels with their core concepts and their inter-relationships. The decomposition into three submodels supports its extensibility. Also, our architectural solutions are organized in a taxonomy of tactics and patterns. New tactics and patterns may be added to the existing ones in order to represent other architectural solutions. In practice, many organizations develop or refine their own architectural solutions. However, the adopting company must be able to associate impacts on quality attributes with the architectural solution.

The following sections discuss the conceptual model in more detail.

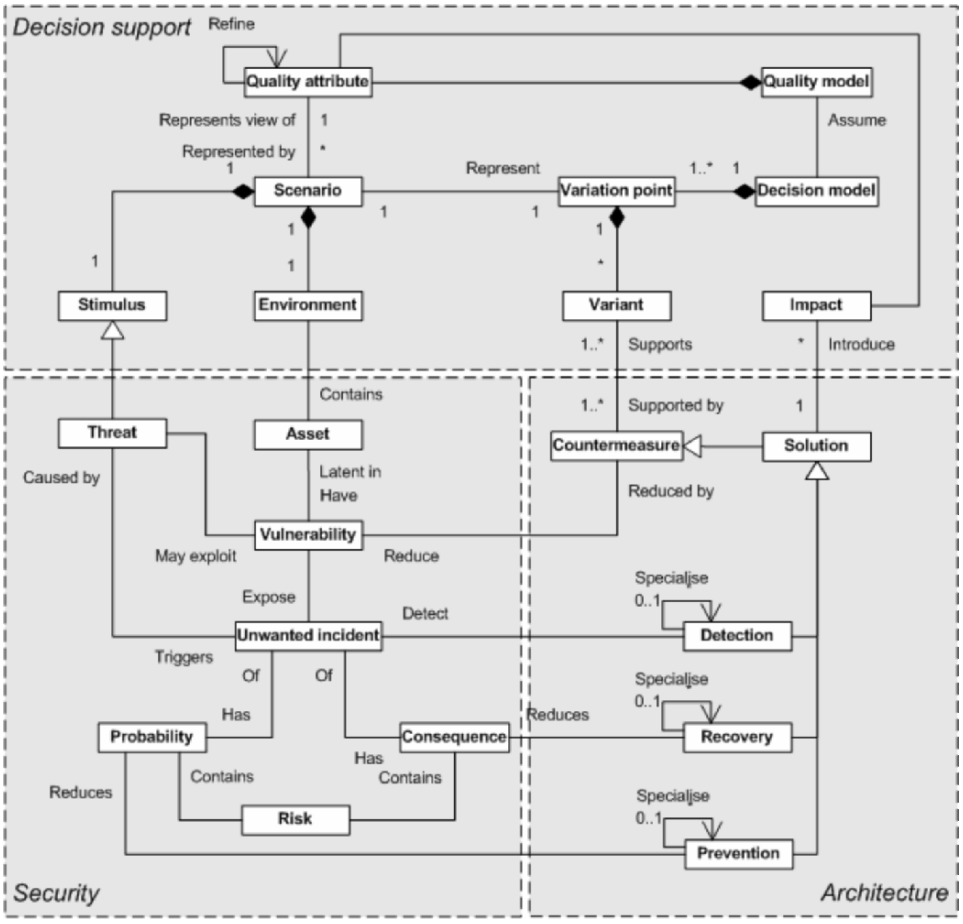

Fig. 8.1. Conceptual model of reference architecture

8.3.1 Security Submodel

The security submodel deals with risk assessment. Core concepts include threat, asset, vulnerability, unwanted incident and risk. Supplementary concepts include probability and consequence.

Arguably, the main objective for any efforts related to security is managing and mitigating risk. This might have significant economical impacts. Return of investment in security efforts must therefore be carefully evaluated [1, 11, 24]. In a product line context, this is even more important. To achieve strategic reuse one depends more heavily on careful planning and design [10].

The proposed reference architecture is not, however, intended to be a tool for risk assessment. Although it does cover the central topics for a risk assessment and provides core support for these activities, the application software architect should carefully consider the

threats, the assets, and their vulnerabilities. Subsequently, a prioritized list of risks can be created, based upon knowledge of the unwanted incidents with their probabilities and consequences. Preferably, a validated methodology should be used to support this process, for example CORAS [62], SAEM [9] or the approach of Cavusoglu et al. [11].

Threats are the raison d'être for all security related requirements. By threat we mean a potential cause of an unwanted incident, which may result in harm to assets (i.e., reduce the asset's value). In the conceptual model above, threats are modeled as a kind of stimuli to a scenario. Thus, the scenarios we consider in terms of security are triggered by threats.

Assets are entities in the software system, such as information, services and components, to which stakeholders assign a value. Different stakeholders are likely to assign different values to the same asset. Assets are exposed to threats.

Vulnerability is a weakness of an asset (or group of assets) that can be exploited by one or more threats. It can also be viewed as weakness in the controls that should protect the asset. Vulnerabilities can be reduced by countermeasures (Sect. 8.3.2).

Unwanted incident is one kind of event that reduces the value of assets. Unwanted incidents occur as a result of a threat exploiting a vulnerability of an asset. Unwanted incidents have a probability, i.e., there is a certain probability that a particular unwanted incident will occur within a given timeframe. The probability is a value between 0 and 1. The value 0 means that the incident will never occur. The value 1 means that the incident will certainly occur. Similarly, each unwanted incident has a consequence. The consequence should be scaled to a value between 0 and 1. This scaling will naturally lead to imprecise numbers, but the benefit of being able to evaluate the consequences for a set of assets will for this kind of context bring significant benefits. Additionally, consequence and probability is used for the assignment of risk – thus bringing benefits in terms of simplified risk assessment and subsequent planning of countermeasures.

We model *risk* as the product of probability and consequence of an unwanted incident. Thus, if either can be reduced to zero, the risk is zero. More likely, the value zero cannot realistically be achieved for either probability or consequence. Rather, the software architect should consider both aspects concurrently in order to obtain a good basis for deciding upon countermeasures. In terms of security architectural design, a prime objective is to construct systems that carefully balance the risk with the economical impact of implementing the countermeasures.

8.3.2 Architecture Submodel

Within the submodel architecture we have located the concepts that are related to the architectural design of the application. These concepts include countermeasure, solution, detection, prevention and recovery.

A *countermeasure* is some kind of action, normally associated with some form of security control (an artifact), which seeks to reduce the vulnerability of an asset (or group of assets). A variant, as discussed in Sect. 8.3.3, is supported by one or more countermeasures. The meaning of this is that a variant is distinct within the variation point if the set of countermeasures is unique among the variants. An asset might be protected by multiple countermeasures, and the mapping of the countermeasure(s) to the architecture model is done to give the best possible effect for the asset in question.

A *solution* is an architectural decision that is used to achieve a quality attribute response. Beneath this definition we include both architectural tactics and patterns. An architectural tactic is a means of satisfying a quality-attribute-response measure by manipulating some

aspect of a quality attribute model through architectural design decisions [3]. Tactics are means, principles, techniques, or mechanisms that facilitate the achievement of certain qualities in architecture. Similar to patterns, tactics capture a way to achieve a certain quality requirement, but are not concrete enough to be used directly and hence have to be instantiated as patterns. Examples of tactics for the quality attribute "reliability" are redundancy and exceptions. Tactics may be specializations of another tactic. At some level of specialization, the tactic becomes a pattern – i.e., a concrete solution to a problem. Referring to Fig. 8.1 above, architectural tactics are a kind of solution that promises to contribute to the wanted response from the scenario (i.e., maintain some of the security qualities discussed in Sect. 8.4). Unwanted incidents have two important properties: a probability and a consequence. We use this interpretation of unwanted incidents to identify three generic security tactics: detection, prevention and recovery (the latter two directly addressing probability and consequence).

These high level tactics are useful for the application designer in order to facilitate reasoning about general approaches to solving the security requirements. However, similar to high level qualities, they have very weak prescriptive power. We need specialized solutions. Figure 8.2 illustrates the high level part of our solutions taxonomy (the security architecture language is illustrated in Fig. 8.6).

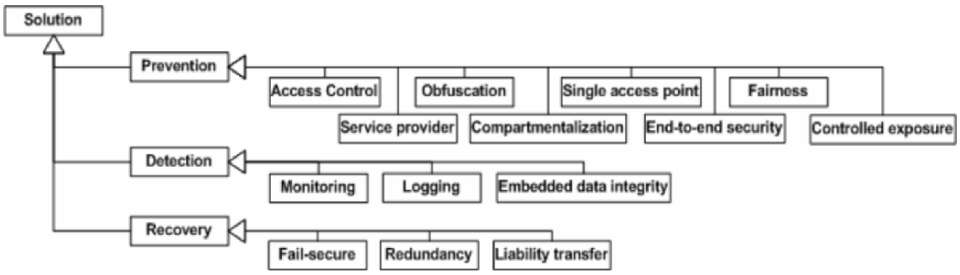

Fig. 8.2. High level solutions hierarchy

Prevention. Prevention tactics are used to reduce the probability of unwanted incidents. Figure 8.2 shows eight different specializations of this tactic that can be used in order to accomplish this, possibly in combination. *Access control* is the implementation of authorization, i.e., the process of ensuring that only designated actors are permitted to perform certain actions on the asset. *Service provider* includes approaches that delegate the implementation of preventive measures to external entities. *Obfuscation* means to re-arrange information in order to make it less intelligible. Cryptography is an example of obfuscation. *Compartmentalization* involves creating multiple security barriers and thereby reducing the probability that an attack endangers the whole system. *Single access* point is exactly the opposite of compartmentalization; it involves centralizing access to the system. The rationale is that it is easier to implement one access point correctly. *End-to-end security* means to ensure security over the whole information chain. For many complex IT systems, this tactic is of key importance as the number of part systems increases. *Fairness* denotes tactics that seek to prevent a single threat agent from taking over the system. Finally, *controlled exposure* is similar to obfuscation but includes mechanisms that actively partition information into a visible and an invisible part. Common for all prevention tactics is

that they do not eliminate the probability of unwanted incidents. Rather, they reduce this probability to a certain level.

Detection. Detection means to determine that something is happening or has happened. It does not affect the system's direct resistance towards an attack. However, the detection tactic can have a great value in many system environments. For example, it may enable continuous improvement of system security. By examining unwanted incidents that have happened, the system can be tuned to counter these kinds of incidents in the future. *Monitoring* and *logging* are two kinds of detection tactics. Monitoring has some kind of active aspect, for example a process that continuously checks for changes or unwanted patterns in the usage of a system. Logging, on the other hand, is primarily a passive arrangement. An example might be the logging of certain events to a file. At some undefined point in time, the log might be inspected. *Embedded data integrity* implies that extra information is added to the original data which can be used to verify tampering.

Recovery. Recovery is the last main group of tactics. It seeks to address security concerns by reducing the consequence (or negative impact) of incidents. Three subgroups of recovery tactics are illustrated. *Fail-secure* is the tactic of designing the system so that in the case of unplanned events it will fail to a secure state, a state in which the system cannot be further jeopardized. *Redundancy* is the tactic of employing multiple, somewhat independently working components with similar functional capabilities in order to withstand certain failures in the component group. Finally, the tactic *liability transfer* involves reducing the consequences for a system by transferring responsibility to another party. Like prevention tactics, recovery tactics are not perfect. They cannot fully eliminate the consequences of unwanted incidents.

As tactics are specialized, they become more prescriptive with respect to architectural design. Further specialized, they become *architectural patterns*, prescribing components,

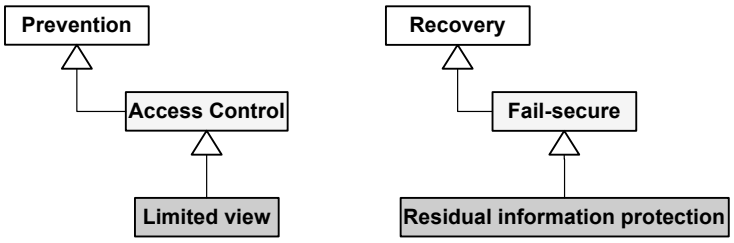

Fig. 8.3. Example specialization

component specifications, component collaborations and component roles. Figure 8.3 illustrates how two patterns (limited view and residual information protection) implement two specialized tactics (access control and fail-secure, respectively). These two are examples from the security architecture language depicted in Fig. 8.6.

Patterns are filled with a gray background in order to illustrate that they are more prescriptive than tactics.

Limited view implies that the user can only see information, menus or options for which he is authorized. That is, access control is performed before information is presented. The opposite approach, called full view with error, implies that access control is performed at a later stage, for example upon trying to execute a menu choice or view detailed information for an item.

Residual information protection involves making sure that no information is left available after a system crash or unexpected application termination. In this way the tactic of secure failure is maintained.

8.3.3 Decision Support Submodel

Within the decision support submodel we group concepts that deal with representing, organizing and reasoning about requirements. It is generic in the sense that it can equally well support other software qualities (e.g., generic ISO 9126 qualities).

The *quality model* represents and organizes our vocabulary for security requirements in a common, easy to use structure. Within a product line, there will be variations in the quality requirements between the different products. The ISO model says the following about security: "Attributes of software that bear on its ability to prevent unauthorized access, whether accidental or deliberate, to programs or data." This definition is clearly too generic to support requirement specifications for software architectures, a view that is also supported by Jung et al. [30]. The quality model we have developed for security is a specialization of the general ISO 9126 model for software qualities [29]. The detailed quality model is presented in Sect. 8.4.

The quality model is broken down into *quality attributes*, each of which is a characteristic of a software product. Quality attributes can be refined, meaning that they have one or more subcharacteristics. Further, quality attributes may influence each other, through the *impact* of architectural solutions.

We use *scenarios* to represent (views of) quality attributes. They consist of the three main elements: environment, stimulus and response.

All scenarios have an *environment*, i.e., a context that may include aspects such as system elements (i.e., assets), actors and processes.

Secondly, the scenario has a *stimulus*. The stimulus is used to model the activation of the scenario, i.e., what triggers the architecture's reaction to a security related concern. Generically, the stimulus may take different forms. In relation to security, a stimulus is something that may compromise the security of the system under evaluation. Thus, we model threat as a kind of stimulus.

Lastly, the scenario has a (set) of *response(s)* that are the *variants*. The response is used to model the architecture's reaction to the stimulus. Openness in the architecture is represented as multiple responses within the same scenario, i.e., some quality aspect that can vary and that has to be resolved by the software architect. The variation point includes a description of the achieved effect on the quality attribute represented by the scenario and a (set of) architectural solution(s) that promise to address that quality attribute. Each response details the architectural solution used to achieve it and other known effects of this architectural decision. Typically, an architectural decision will have impacts on other nonsecurity related quality attributes. The application architect must determine how to prioritize these.

Quality attribute: Confidentiality → Withstand attacks in a group of cooperating applications

Environment: Application a_c provides a set of services that makes sensitive information available to other collaborating applications over the Internet.

Stimuli	Response	Resolution
An application a_m attempts to invoke services from application a_c without the required authorizations.	The architecture prevents a_m from accessing nonauthorized services from a_c.	**V. 1**
	The architecture allows a_m access to the services, but all accesses are logged. This facilitates recovery.	**V. 2**

Scenario resolution:

Ref.	Approach	Architectural solution
V. 1	The architecture requires that a_m is both authenticated and authorized before being allowed access to the services.	Prevention. Access control. (Component) authentication. Authorization.
V. 2	The architecture acknowledges that availability of information may be more critical than preventing access to it. However, by logging all accesses to the information, liability is put on the application a_m.	Recovery. Liability transfer. Digital certificates. Auditing.

Each alternative solution included in the variation point is denoted a *variant*. Variants implement the architectural solutions indicated in the scenario.

Typically, an architectural decision affects more than one quality attribute. For example, the same architectural pattern can improve performance but cause an increase in complexity and a reduction of maintainability. We denote this phenomenon *impact*. Impacts are summarized for each architectural solution.

Lastly, the *decision model* is the collection of scenarios. The software architect must first determine the quality requirements that apply to the application to be designed. Then, the scenarios representing these quality requirements must be identified. These scenarios will then constitute the decision model for that application. Subsequently, for each applicable scenario, the variation point is resolved in light of the particular requirements of the application. The decision model is presented in detail in Sect. 8.5.

An example scenario, representing a certain aspect of confidentiality (maintaining confidentiality in an application integration setting) is given above. In this example, the scenario has two distinct responses – representing two alternative ways to affect the quality attribute. One is to prevent the incident from occurring, i.e., reduce its probability through access control. The other is to reduce the consequence of the incident by transferring liability. Each of the two variants is subsequently described in more detail in the scenario resolution part. The tactics and patterns that will help the architect in reaching the desired effect by resolving the variation point are documented in the column "Architectural solution."

The scenario is presented in two main parts. The main part, on the top, contains the environment, stimuli and response. The second part describes how the response can be accomplished. This scenario encompasses two different architectural decisions; (V.1) is to reduce the probability of a security breach or (V.2) is to reduce the consequences.

The scenarios included in this reference architecture have been developed by extracting security requirements from a number of companies, refining them to conform to our scenario structure and then collectively reviewed in order to extract generic architectural knowledge. Additionally, security literature has been utilized to support and extend this knowledge [21, 22, 28, 47, 54].

8.4 Quality Model

This section describes our quality model. It is used to represent and organize our vocabulary for security requirements The quality model assumed in the reference architecture is a specialization of the general ISO 9126 model for software qualities [29] (Fig. 8.4).

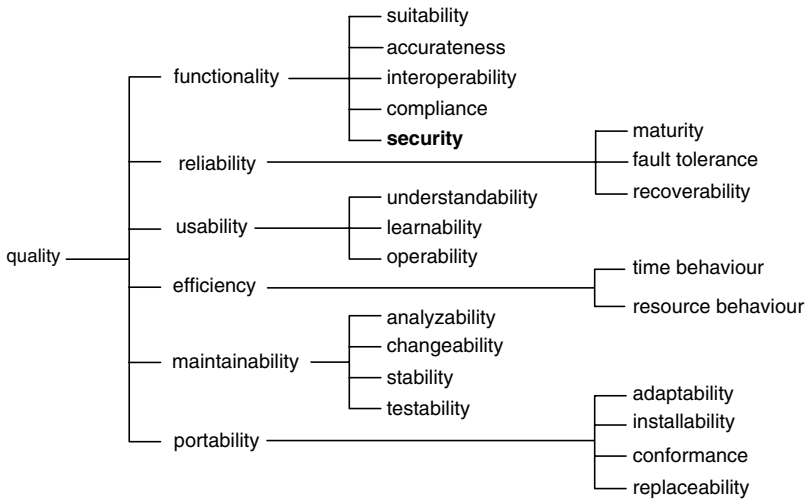

Fig. 8.4. The ISO9126 quality model for software systems

Although useful at an overall level, the quality "security" from ISO 9126 is too vague to be useful in requirements engineering. Furthermore, Jung et al. show that security as a subcharacteristic of "functionality" is problematic [30]. By defining security using more concrete terms, this problem can be reduced. To precisely capture and support reasoning about security requirements, security is broken down into the four intermediate level security quality attributes *integrity*, *confidentiality*, *availability* and *accountability*. Figure 8.5 illustrates the specialization relationship.

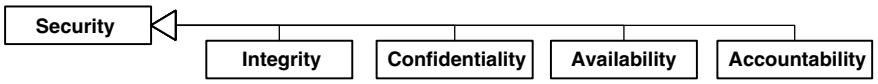

Fig. 8.5. Security quality breakdown

We understand with these four attributes the following:

1. Integrity: The property that assets have not been altered or destroyed in an unauthorized manner
2. Confidentiality: The property that assets are not made available or disclosed to unauthorized actors
3. Availability: The property of an asset being accessible and usable upon demand by an authorized actor
4. Accountability: The property that ensures that the actions of an actor may be traced uniquely to that actor

Our experience is that this breakdown is very useful as it assists in the determination of security requirements. For example, during risk assessment, it helps improving common understanding among the participants. By considering each of the four generic security quality attributes in turn, one can more easily determine what quality properties are relevant or not for the particular application. As a trivial example, in the context of an application that serves public information via the Internet, confidentiality might be a low priority quality, but integrity is likely to be of high importance.

As the complexity of the security domain is fairly high, we have used these four quality attributes as a starting point for generating and organizing more specific business requirements related to those aspects of security. Some examples are given in the Tb. 8.1.

Table 8.1. Examples of business requirements

quality	business requirement
integrity	secure use of third party components
	protection against unauthorized manipulation in user's application
confidentiality	withstand attacks in a group of collaborating applications
	maintaining security on shared computers
availability	protection against service disruptions
accountability	prevent false impersonation

For each business requirement there are multiple scenarios exemplifying the requirement. The scenarios are documented in the decision model (presented in Sect. 8.5).

Although we do not discuss the requirement specification process in more detail here, we assume that the application designer is supported by a risk assessment of the system. In security engineering, precise requirements can only be made after assessing the risks pertaining to the assets encompassed by the system. Broadly speaking, risks are events that can jeopardize the security qualities. It is important that the risk assessment is done at a suitable level of detail so as to give a good understanding of which assets are worth protecting and the level of risk that each of these assets is exposed to. This creates the basic decision framework for the application designer when determining the security qualities that shall apply for the application.

8.5 Decision Model

The process of building software architectures involves making well-considered design decisions that are highly sensitive to problem domain knowledge. This section contains a decision model for software architectures where security requirements must be addressed. The model has been developed in cooperation with four industrial partners. 1–2 representatives from each company participated in 2–5 one-day meetings discussing and capturing current practices. Roughly 150 man-hours were used on this activity. Additionally, material from security literature was used to aid the model's development. In order to reduce redundancy, literature is primarily cited in the security architecture language (Sect. 8.6), where the architectural solutions providing support to the scenarios have been described.

The structure of the decision model follows the conceptual reference architecture illustrated in Fig. 8.1; for each scenario, an environment describes the overall context, e.g. the considered assets and the kind of applications that might be applicable for the scenario. Subsequently, the stimulus illustrates the threat towards the assets.

Now, there might be different approaches to resolving a scenario. These are essentially variants within the product line architecture. Different architectural solutions illustrate design approaches that address the threat. Many variants thus refer to multiple architectural solutions (the numbers refer to the solutions presented in the security architecture language, the topic of Sect. 8.6). However, each response brings its own effects in terms of how the risks are affected and effects on other quality attributes.

8.5.1 Integrity

In the context of security, integrity is the property of information that it has not been manipulated by unauthorized actors. Integrity may be threatened wherever information is stored, transmitted or used. We have identified three relevant classes of environments within the scope of this quality model (a) third party components, (b) dynamic security boundaries, and (c) unauthorized manipulation in user's application.

Using Externally Developed Components Securely

The use of third party components (COTS or open source components) is an attractive approach to reduce cost of developing software. However, the approach brings significant

challenges with respect to security. For example, it is difficult to determine whether a third party component contains dangerous code or not. Third party components cannot, without additional efforts, be trusted to the same degree as internally developed components. For the distrustful, a third party component has the same security characteristics as a virus. In real life however, most third party components will include some kind of insurance.

Scenario I.1 – Third Party Components

Environment: A software system includes components developed outside the company. Such components cannot always be trusted to the same degree as components developed within the company. Incurred threats include Trojan horse attacks and espionage [35].

Stimuli	Response	Resolution
A third party component attempts to modify information it is not authorized to modify.	The architecture prevents the attempt by restricting third party components' freedom, thus reducing the consequence of the attack.	V.1
	The architecture prevents the attempt by enforcing traditional access control policies for third party components.	V.2
	The architecture reduces the consequence of the attack by imposing liability onto the third party component's supplier.	V.3

Scenario resolution:

Ref.	Approach	Arch. Solution
V.1	The architecture reduces the freedom of execution of third party components. Third party components are not allowed to execute potentially dangerous actions.	Pattern 27: Sandbox. Pattern 21: Multi Barrier Security. Pattern 17: Layering.
V.2	Third party components' information access is protected by access control and unauthorized manipulation of information is prevented.	Pattern 3: Authentication. Pattern 6: Authorization.
V.3	As the third party components are verified by the supplier, the supplier can be more easily held responsible for the components' behavior.	Pattern 12: Code Signing.

Maintaining Integrity in Mobile Systems with Dynamic Security Boundaries

Occasionally, the security boundary of information changes over time. This introduces additional requirements with respect to the security architecture.

Scenario I.2 – Tampered Information During Mobile/Offline Usage

Environment: Support for disconnected clients complicates security. One must ensure that the extended system boundary is accounted for by the security architecture. There is a potential for loading the client or server with information that is not valid, e.g., it might have been tampered with.

Stimuli	Response	Resolution
A threat agent may compromise a mobile device and may subsequently cause manipulated information to be transferred to the server system.	The architecture detects maliciously manipulated information.	V.1
	The architecture prevents information from being maliciously compromised.	V.2

Scenario resolution:

Ref.	Approach	Arch. Solution
V.1	Integrity check in the information enables the detection of the maliciously manipulated information and may be used to prevent it from being accepted.	Pattern 12: Code Signing. Pattern 19: Message Authentication Codes.
V.2	The mobile device prevents successful attacks from threat agents.	Pattern 3: Authentication. Pattern 8: Biometric Authentication.

Scenario I.3 – Tampered Code Mobile/Offline Usage

Environment: Mobile clients get their code installed from the central server. In case the server is compromised, there is a potential for loading the client with code that is not valid, e.g., it might have been tampered with.

Stimuli	Response	Resolution
A threat agent has compromised the application server and thereby threatens to provide the client device with potentially dangerous code.	The architecture prevents maliciously manipulated code from being accepted by the client device.	V.1

Scenario resolution:

Ref.	Approach	Arch. Solution
V.1	Code on the server is protected by authenticity measures. Any integrity violations are detected by the client.	Pattern 12: Code Signing.

Maintaining a Defense Against Unauthorized Manipulation in User's Application

Information is, and should be, easily available in the user's application. However, ease of access constitutes a security risk in its own right.

Scenario I.4 – User Leaves Computer for some Time

Environment: A computer is left unattended for some time leaving it exposed to unwanted incidents.

Stimuli	Response	Resolution
A threat agent attempts to exploit a computer that has been left unattended by its user.	The architecture prevents any incidents by locking the application after a certain elapsed time, requiring the user to log in afterwards.	V.1

Scenario resolution:

Ref.	Approach	Arch Solution
V.1	The architecture enables the locking of the application after a certain period of inactivity.	Pattern 31: Timeout. Pattern 3: Authentication.

Scenario I.5 – Application Used on Exposed Device

Environment: An organization may gain significant benefits by enabling mobile workers to use corporate applications while being on the move. However, the use of mobile computers in noncontrolled physical environments may give rise to additional risks towards integrity of information.

Stimuli	Response	Resolution
A threat agent has gained physical access to the mobile computer.	The architecture helps in detecting the attack and providing timely alarms to the user.	V.1
	The architecture helps prevent unwanted incidents by strengthening user authentication procedures when the application is used in a hostile environment.	V.2
	The architecture reduces the consequence of unwanted incidents by reducing the amount of information stored on the mobile computer.	V.3
	The architecture reduces the consequence of unwanted incidents by limiting the information access rights when the application is used in a hostile environment.	V.4

Scenario resolution:

Ref.	Approach	Arch. Solution
V.1	The architecture contains surveillance functionality that detects hostile behavior.	Pattern 16: IDS (Intrusion Detection System). Pattern 1: Anomaly Detection.
V.2	The architecture supports the implementation of contextual security policies by which requirements on authentication can be adjusted.	Pattern 4: Authentication Levels.
V.3	The architecture minimizes the amount of information stored on the mobile computer.	Pattern 30: Thin Client. Pattern 25: Residual Information *Protection.*
V.4	The architecture supports the implementation of contextual security policies by which authorization levels are adjusted depending on the physical environment.	Pattern 14: Contextual Authorization.

Scenario I.6 – Information Access Rights not reflected in Application

Environment: During the development of applications, there is a continuous danger that the correct authorizations are not reflected in the application.

Stimuli	Response	Resolution
Security sensitive application code is being written or maintained.	Accidental errors, causing breaches in the integrity of the information, are prevented in the application code.	V.1
	The developer is provided with instruments to reduce the likelihood of not reflecting the correct authorizations.	V.2

Scenario resolution:

Ref.	Approach	Arch. Solution
V.1	Access rights are encapsulated with the information thus reducing the risk of writing erroneous application code.	Pattern 18: Limited View.
V.2	The architecture assists the application developer in maintaining the integrity of information. The architecture prescribes a common security resolving functional component.	Pattern 24: Reference Monitor. Pattern 28: Sentry.

8.5.2 Confidentiality

Confidentiality is the ability of a system to restrict access to information to authorized users only.

Withstanding Attacks in a Group of Cooperating Applications

In systems composed of multiple cooperating applications, a certain level of trust must be present. However, it is important to determine the implications and possible actions that should be the result of potential breaches of this trust [48].

Scenario C.1 – Application Integration

Environment: Application a_c provides a set of services that makes sensitive information available to other applications over the Internet.

Stimuli	Response	Resolution
An application a_m attempts to invoke services from application a_c without proper authorizations.	The architecture prevents a_m from accessing nonauthorized services from a_c.	V.1
	The architecture allows a_m access to the services, but all accesses are logged. The consequences for a_c are therefore reduced.	V.2

Scenario resolution:

Ref.	Approach	Arch. solution
V.1	The architecture requires that a_m is both authenticated and authorized before being allowed access to the services.	Pattern 3: Authentication. Pattern 6: Authorization.
V.2	The architecture acknowledges that availability of information may be more critical than preventing access to it. However, by logging all accesses to the information, liability is put on the application a_m	Pattern 3: Authentication. Pattern 2: Auditing. Pattern 15: Digital Signatures.

Maintaining Security on Shared Computers

The benefit of being able to use your applications despite being without a private computer might be justified in certain circumstances. However, the requirement to maintain the confidentiality of information is severely stressed when the computer used to access the application is shared with other people who may constitute potential threat agents.

Scenario C.2 – Secure Use on Non-private Computers

Environment: After a service has been used from public Internet access computers, there may be traces of information left on the machines that can cause confidentiality violations. This is a serious threat to confidentiality, as systems and applications may crash, which could prevent full application control of the data/code loaded onto the computer. Further, the administrative routines for the computer could prevent the user from manually ensuring that traces of service usage have been cleaned up.

Stimuli	Response	Resolution
A user accesses a corporate application from an untrusted computer. A threat agent subsequently gains control of the computer.	The architecture eliminates the storage of potentially vulnerable information on the untrusted computer.	V.1
	The architecture protects all information stored on the untrusted computer.	V.2
	The architecture forbids access to particularly sensitive information while using public computers.	V.3

Scenario resolution:

Ref.	Approach	Arch. Solution
V.1	Information is not persistently stored on the client computer, it is only viewed. The central server maintains all information.	Pattern 13: Cookie. Pattern 30: Thin Client.
V.2	All information that is stored, either temporarily or permanently is protected and only readable by authorized actors.	Pattern 11: Cryptography (of all sensitive data in memory and persistent storage). Pattern 25: Residual Information *Protection.*
V.3	When an application is used on untrusted computers, only certain, nonsensitive information is available.	Pattern 14: Contextual Authorization

8.5.3 Availability

Availability is a system's ability to provide service for a given percentage of the time. Understandably, a system that is unable to provide service may cause great disadvantages for its stakeholders. Reducing the availability of a service may of course be in the interest of a threat agent. Thus, it is critical that availability is included as part of a security quality. We choose to discuss availability in terms of the timeliness of services.

At a high level, there are two principal causes for reduced service timeliness (a) the service proper or (b) the service access path (i.e., networks and associated infrastructure).

For a client however, it is impossible to distinguish between the two. We also address the problems of hardware sabotage, i.e., attacks that cause physical damage to hardware.

Avoiding Service Disruptions

A threat agent may gain significant benefits from disrupting a service. Depending on the potential negative business impact, the software architecture should support instruments to reduce the likelihood or reduce the consequence of such attempts.

Scenario Av.1 – Malicious Third Party Components

Environment: When using a third party component in a solution, there is a threat that the component may attempt to cause service disruption.

Stimuli	Response	Resolution
A third party component attempts to cause service disruption.	The architecture prevents the component from disrupting the service.	V.1
	The architecture reduces the consequence of unwanted incidents caused by the component.	V.2

Scenario resolution:

Ref.	Approach	Arch. solution
V.1	The third party component is prevented from doing certain operations that may cause disruptions of the service.	Pattern 27: Sandbox. Pattern 21: Multi Barrier Security.
V.2	The component causes disruption at one server, but redundancy ensures that the service is quickly restored (note: measures need to be taken to ensure that the same incident does not occur at the redundant server).	Pattern 10: Clustering. Pattern 20: Mirror Sites.

Scenario Av.2 – Denial of Service Attacks

Environment: A threat agent may establish Denial of Service Attacks towards a service provider. Many sophisticated attacks, such as Distributed Denial of Service Attacks, may be difficult to distinguish from the load caused by high popularity among a high number of clients.

Stimuli	Response	Resolution
A DoS attack is established against the system.	The architecture reduces the consequence of the unwanted incident.	V.1
	The unwanted incident is detected and causes system administrators, etc. to be warned.	V.2

Scenario resolution:

Ref.	Approach	Arch. solution
V.1	The architecture reduces the impact of DoS attacks by invoking load balancing techniques that prevent the system from halting completely.	Pattern 26: Resource Throttling, Pattern 7: Bandwidth Throttling.
V.2	The architecture enables the detection of the attack.	Pattern 16: IDS (Intrusion Detection System). Pattern 1: Anomaly Detection.

Scenario Av.3 – Denial of Service Attacks Towards a User

Environment: As a security measure, an actor's account may be protected by a maximum number of failed authentication attempts. After this, the account is disabled for some time to prevent misuse. A threat agent may exploit this fact, and establish Denial of Service Attacks towards a single (or group of) user(s).

Stimuli	Response	Resolution
A threat agent establishes a DoS attack against a user of the system by attempting to login several times, thus exceeding the user's allowed number of failed login attempts.	The exposure of the login ID is reduced.	V.1
	Login IDs are not explicitly open for attack.	V.2

Scenario resolution:

Ref.	Approach	Arch. solution
V.1	The attack is not prevented, but the architecture should reduce the exposure of users' login IDs to minimize the chance of such attacks.	Pattern 18: Limited *View*.
V.2	The architecture supports the use of smartcards for authentication, optionally combined with biometric authentication, which do not expose login IDs to attackers. Alternatively, password authentication may be the last mechanism in a combination.	Pattern 3: Authentication. Pattern 8: Biometric Authentication.

8.5.4 Accountability

Accountability is the obligation or willingness to accept responsibility for one's actions. That is, accountability enables us to place trust in actors and have reasonable expectations about actors behaving according to their responsibilities. [1]

There are two key objectives related to accountability (a) making sure that the actor is identified correctly and (b) making sure that the identified actor cannot deny having performed certain actions. The text below contains scenarios that address the first of these two objectives. Scenarios representing the second objective have been omitted due to space limitations, but are typically resolved using some form of digital signature (see Pattern 15: Digital Signatures).

Preventing False Impersonation

An actor (e.g., a person or a component) should not be able to use the identity of another actor in a false manner.[2] If the attack is successful, the actor may subsequently endanger the confidentiality, integrity and availability of a target system's assets.

Scenario Ac.1 – User Authentication

Environment: Internally, most software systems need the ability to represent actors with their identity. In order to ensure a suitable level of trust in the identity the actor must be authenticated.

Stimuli	Response	Resolution
A threat agent attempts to impersonate an application by guessing valid username and password combinations.	The architecture prevents unwanted incidents by enforcing better policies for usernames and passwords, making it much more difficult to guess them.	V.1
	The architecture detects the attack and activates an alarm to administrative personnel.	V.2
	The architecture prevents that simply guessing a valid user ID and password combination is enough to breach the authentication procedure.	V.3

[1] Accountability includes a range of other issues also, of course. In order to enforce this quality, involved actors must have a common framework to deal with representations, negotiations, legal principles and resolution mechanisms. The framework should be maintained by an independent governing authority.

[2] Note that impersonation is a commonly used phenomenon in distributed systems because it allows a component to act on behalf of a user or another component.

Scenario resolution:

Ref.	Approach	Arch. Solution
V.1	The architecture reduces the likelihood of successfully guessing valid usernames and passwords.	Pattern 5: Authentication Policy. Pattern 22: Password.
V.2	The architecture detects the attack through monitoring functionality and reports the attack to a security team.	Pattern 16: IDS (Intrusion Detection System).
V.3	The architecture enables the use of a combination of different authentication mechanisms, for example biometric authentication.	Pattern 8: Biometric Authentication.

Scenario Ac.2 – Integration of Authentication Systems

Environment: Useful software systems are long lived. Such systems are thus more likely to face a requirement to be integrated with other systems. Application integration is a complex problem area which also raises concerns about security. Each application may have a separate security architecture dealing with authentication. Integration of these authentication mechanisms, for example by using single sign-on technologies (see Pattern 29: Single Sign-On), is a benefit for efficiency and operability, but might cause potentials for unwanted incidents in the form of more extensive attacks. Once inside, all systems comprising the solution may be compromised.

Stimuli	Response	Resolution
A threat agent has established an attack to impersonate an integrated authentication infrastructure.	The architecture reduces the consequence of the false impersonation attack.	V.1
	The architecture supports detection of the false impersonation attack.	V.2
	The architecture prevents the false impersonation attack.	V.3

Scenario resolution:

Ref.	Approach	Arch. solutions
V.1	The architecture enables the implementation of flexible security policies, for example including multiple authentication levels. There might be good reasons to implement policies that include extra authentication procedures for specific, highly critical tasks, even if the actor has already signed in successfully in the integrated authentication infrastructure.	Pattern 4: Authentication Levels. Pattern 14: Contextual Authorization.

V.2	The architecture detects that an actor has impersonated the system by observing unusual behavior by the actor.	Pattern 16: IDS (Intrusion Detection System). Pattern 1: Anomaly Detection.
V.3	The architecture enforces the use of stronger authentication mechanisms that are judged to be strong enough for all systems participating in the solution.	Pattern 8: Biometric Authentication.

8.6 Security Architecture Language

One way to structure architectural solutions is according to the kind of tactics they specialize or implement. In the context of security, we have identified and structured a number of architectural solutions. We have structured them according to the three high-level tactics detection, prevention and recovery. We cannot claim that this structure is the only one which is useful because architectural solutions will also address requirements other than security. However, the security architecture language presented here enables application architects dealing with those kinds of requirements to effectively identify tactics and patterns that address particular requirements related to security. The term "language" is discussed in the Chap. 7.

Architectural solutions are not described to a great level of detail. That would be outside the scope of this work. However, we provide references to further documentation. We also document significant impacts on nonsecurity related quality attributes, such as complexity and performance, for each pattern.

8.6.1 Tactics

This section discusses architectural tactics, general solutions adhered to in software architecture. Although this chapter only focuses upon the solutions used to ensure security qualities, such solutions apply to most architectural elements. Architectural solutions are essentially structured knowledge, helping software architects to think in terms of solutions for recurring problems.

We introduced architectural solutions in Sect. 8.3. Tactics can be structured in hierarchies where high-level tactics form the basis for more specialized tactics. At some level of specialization, the tactic is so refined that it takes the form of a pattern.

In this section, we describe three high-level security tactics; prevention, detection and recovery. Below these three, we identify subcategories of tactics. Figure 8.2 illustrates the conceptual hierarchy of security tactics that we discuss in this section. The intention of the figure is to show how we consider the relations between the tactics.

Prevention

Prevention is the tactic of creating barriers that potential enemies cannot circumvent. There will never be fully secure systems, so prevention tactics aim at reducing the probability of successful attacks.

Access control. Access control is the tactic of controlling which actors are allowed to operate upon assets.

Service provider. The tactic of using autonomous, possibly external entities to perform some predefined function on behalf of other actors. Examples might be validators that inspect certain data according to given rules.

Obfuscation. Obfuscation is the tactic of making data difficult to understand, thus increasing the cost of an adversary to interpret the data. This can for example be accomplished by applying cryptographic techniques.

Compartmentalization. The compartmentalization tactic means dividing a system into sections, so that breaking into one section does not enable direct access to the others. This tactic is useful to prevent attacks from damaging the whole system.

Single access point. This tactic is the opposite of the compartmentalization tactic. The single access point tactic is based upon the assumption that it is easier to build *one* good security barrier than multiple ones.

Fairness. The tactic of maintaining fairness involves balancing the consumption of resources fairly according to the current availability. The fairness tactic is a key to reduce the effects of Denial-of-Service attacks for example.

Controlled exposure. Complexity found in many software systems motivates the use of architectural tactics that reduce the risk of revealing information to nonauthorized actors. This tactic states that information is not exposed unnecessarily.

End-to-end security. End-to-end security is the tactic of ensuring that the whole channel between any two actors involved in service exchange is secured. The main rationale is that it simplifies security management. The tactic makes it more difficult to establish man-in-the-middle attacks, and it reduces the likelihood of weak points in the communication chain between two partners.

Detection

Detection is a key tactic in security for determining that the system is under attack. If enemy attacks are detected, they can be countered or knowledge about the attack may be used in the process of improving a system's security. The sophistication of attacks will increase. Therefore, it is crucial to be able to learn from attacks and adapt to an evolving threat landscape. Worth noting is that detecting anomalies is inherently difficult – even the best detection methods will always have false positives or false negatives, or both.

Monitoring. By monitoring, we mean inspecting certain parameters periodically. The monitoring tactic is useful for detecting security attacks as it enables rapid detection of attacks.

Logging. The tactic of logging implies that data are collected for inspection later.

Embedded data integrity. This tactic involves associating extra information with the data in order to facilitate the detection of security breaches. Examples are message authentication codes, extra data added to verify the message's integrity and watermarking.

Recovery

As a high-level security tactic, recovery involves reducing the negative consequences of attacks. In some situations, it makes sense to accept that attacks are successful and rather spend efforts in reducing the consequence of those attacks.

Fail-secure. This tactic implies that in the event of a failure, the system should be left in a secure mode, e.g., without leaving assets unprotected. A main element of the fail-secure tactic is that it is automatic, i.e., the system itself is designed to implement this tactic. One example of this tactic might be the following: If an application or component fails, sensitive data should be left in a protected state. Another example: Avoiding too much detail in error messages; detailed error messages could help attackers to exploit vulnerabilities in the application/component. Detailed error information could instead be written to the audit log.

Redundancy. Redundancy is the tactic of using multiple components with similar functional capabilities in order to withstand certain failures in the component group.

Liability transfer. Liability transfer is the tactic of making someone else responsible for the potential damage.

In the figure below (Fig. 8.6), we illustrate the full taxonomy of architectural solutions. Solutions become more and more specialized towards the leaves of the tree. Thus, tactics are located at the top of the tree, while patterns occur further down. With consideration to the volume of this chapter, only the architectural patterns that are referred to by scenarios of the decision model in subchapter 5 are described. These are indicated with italics in the figure.

8.6.2 Patterns

The following sections briefly present the architectural patterns for security that are referenced by the scenarios in the decision model (marked using italics in Fig. 8.6). Most of the included patterns are nevertheless well documented in the literature [16, 21, 37, 38, 42, 47, 50, 54, 59]. Patterns are more prescriptive than tactics and are therefore described at a more technical level than the tactics. Patterns implement tactics. However, there is a continuum of specialization among architectural solutions. No line can be drawn that unambiguously distinguishes tactics from patterns. The classification presented here is therefore suggestive. Patterns are presented in alphabetical order.

Pattern 1: Anomaly Detection

Architectural solution: Detection → Monitoring → Intrusion Detection System (IDS)

Problem. Attacks via the network, e.g., the Internet, constitute a continuous threat to networked computers (similar to the problem addressed by IDS in general, see Pattern 16: IDS). A key challenge concerning guarding against these attacks is the difficulty in determining what constitutes an attack. It must be expected that threat agents will frequently change the way attacks are established.

Solution. Anomaly-based intrusion detection is a kind of intrusion detection pattern. In this pattern, behavior in the system is analyzed, and unusual behavior is regarded as an attack [37]. The claimed benefit is an increased ability to prevent newly created attacks.

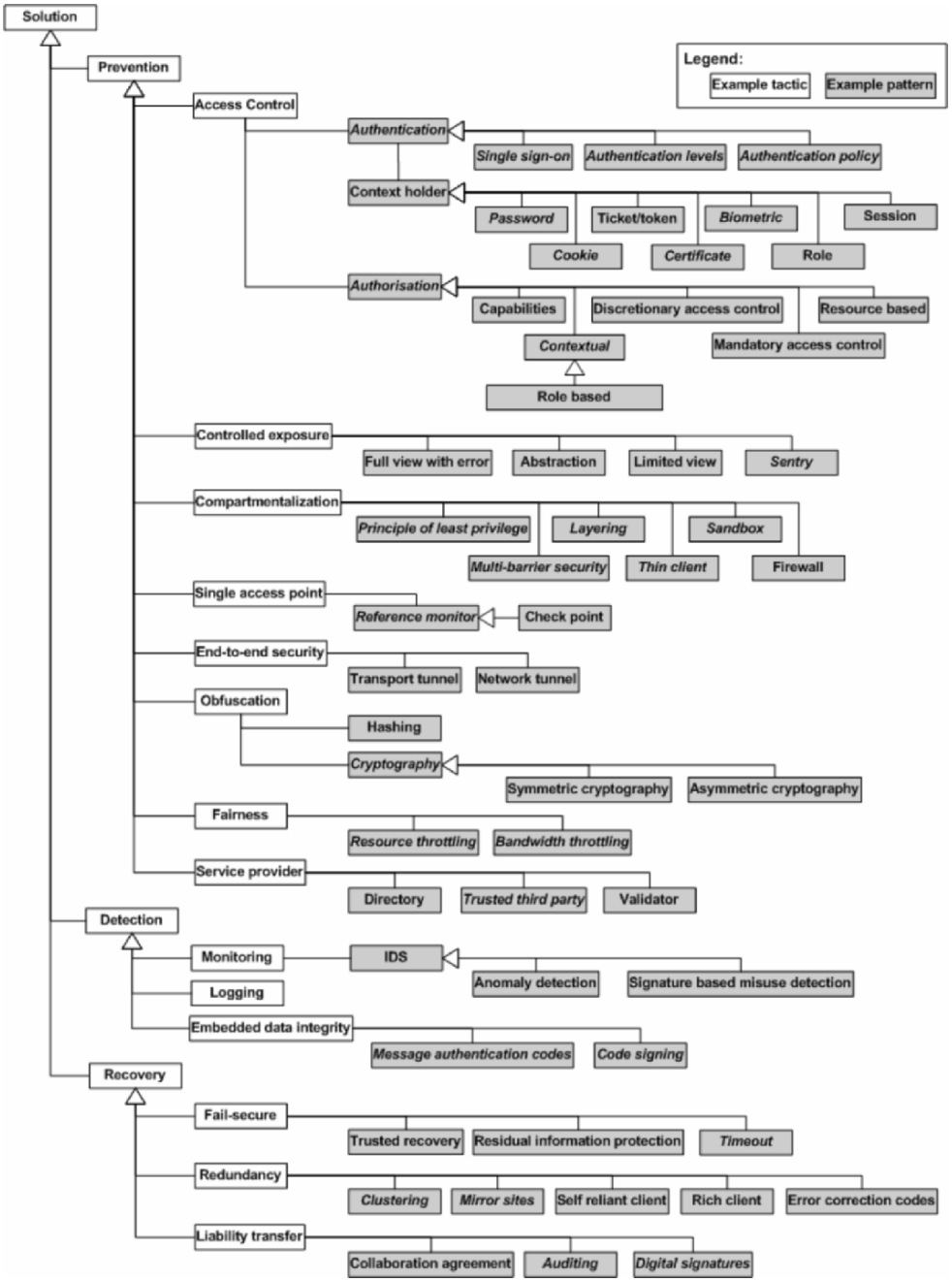

Fig. 8.6. Security architecture language

Impact.

1. Similar to signature based misuse detection IDS; anomaly detection incurs performance overheads, normally even heavier overheads than signature based misuse detection IDS. Their flexibility is also somewhat lower because it takes more effort to reconfigure the rules of the IDS.
2. Operability is negatively affected as these kinds of IDS have a tendency to generate large numbers of false alarms [11].
3. However, anomaly detection has a higher probability of detecting new kinds of attacks, because IDS of this kind are based on a more abstract set of trigger rules [37].

Pattern 2: Auditing

Architectural solution: Recovery → Liability transfer

Problem. Not all actors in a system can be expected to accept responsibility for the actions they have initiated. A mechanism to establish this responsibility is thus required.

Solution. Auditing builds evidence in the correlation of what subject was responsible for a particular event or set thereof. Normally, auditing depends on some kind of logging. These logs contain records of which actor performed certain actions in the system. Additionally, information related to the time of the event, certain communication parameters etc., are recorded. The auditing pattern means managing and using these logs in a controlled manner in order to support placement of responsibility in situations where this is necessary. See also [47].

Impact.

1. Auditing may involve sophisticated log analysis which in turn may have significant resource requirements causing delays for other current processing on the system. These delays may be somewhat alleviated by performing the analysis in low-priority processes.
2. Auditing may have a preventive effect in terms of security. If users of the system know that auditing mechanisms are in place, they might be discouraged from establishing attacks or trying to violate security in the system.

Pattern 3: Authentication

Architectural solution: Prevention → Access control

Problem. In order to implement access control, i.e., ensuring that only authorized actors are allowed to do certain operations, it is crucial that we know the identity of the actor we are dealing with.

Solution. Authentication helps to increase the probability that we allocate the correct identity to the actor. It achieves this by comparing some kind of extra evidence with the supplied identity from the actor.

There are many kinds of authentication solutions available, but there are three main categories (a) Knowledge-based, (b) object-based, or (c) ID-based (i.e., based upon something you know, something you have or some feature of yourself, respectively). Smartcards are able to support efficient and strong authentication [47], but require that the terminal or computer is equipped with a smartcard reader.

Authentication can be tailored to suit the security requirements of the application in various ways. That is, the probability of correct authentication is given by how authentication is implemented, for example using the length and alphabet of passwords as parameter (longer, more complex passwords are harder to guess). Please refer to [42] for a thorough discussion of the topic.

Impact.

1. Multiple aspects of security benefit from (and depend upon) authentication. Primarily, accountability cannot be attained without sufficiently strong authentication of the actor. Secondly, as access control depends upon authentication as a basic element, confidentiality and integrity will indirectly benefit.
2. Usability. Authentication must be done in a manner that is suitable for the environment in which the actor operates. Otherwise, usability will suffer. Usability is highly dependent on the choice of authentication solution (see above). Also, different user contexts will favor different authentication solutions. A range of usability design tactics that can assist in accommodating different authentication solutions can be found in [58].

Pattern 4: Authentication Levels

Architectural solution: Prevention → Access control → Authentication

Problem. For some applications, a single strength of authentication is inadequate. There might be different usage scenarios, caused by different user situations, which mandate differentiation.

Solution. Depending on the criticality of the asset to be secured, different strengths of authentication may be applied [16].

Impact.

1. In general, the same security benefits as for Pattern 3: Authentication apply. However, by differentiating the authentication levels, the security qualities will be improved as it simplifies authentication tasks for the user and thus may stimulate the use of better password routines, for example.
2. Operability will be negatively affected. Administration of different authentication levels requires good configuration practices and tools.
3. Usability will generally be positively affected. Users are able to do more with less hassle. Usability may also be negatively affected if the need for additional authentication procedures is not clearly motivated and communicated.

Pattern 5: Authentication Policy

Architectural solution: Prevention → Access control → Authentication

Problem. Many technologies, authentication technologies being no exception, do not give the promised benefits unless they are part of an overall policy for their use. The planned use of different authentication technologies should be coordinated and executed in order to achieve goals set by the organization.

Solution. An authentication policy can include multiple aspects, depending on the authentication technologies being used. Examples are (a) limited number of login attempts, (b) increasing delays between allowed login attempts, (c) password strength policies, and (d) invocation of different authentication levels (described above). See also [12].

Impact.
1. Given good tools for configuration and maintenance, operability is affected positively.
2. Accountability is highly dependent upon proper authentication of actors. Authentication policies can thus improve accountability.

Pattern 6: Authorization

Architectural solution: Prevention → Access control

Problem. Security implies that a certain level of control can be exercised with respect to which actors are allowed to perform operations in the system. In other words, we want to control which subjects are allowed to perform which operations on which objects. Implementing this kind of control function is nontrivial.

Solution. Managing relationships of the type (subject, operation, object) is called authorization management [50]. This is a complex problem area as its solution is heavily dependent upon many characteristics. Example characteristics that influence the solution are (a) the number of subjects compared to the number of objects, (b) the usage of the assets (i.e., the objects), and (c) the frequency of changes to the authorizations. Pattern 14: Contextual Authorization discusses a specialized alternative authorization regime. See [41, 44, 50, 55] for other variations of authorization schemes.

Impact.
1. The security qualities integrity and confidentiality can only be satisfied by a proper implementation of authorization.
2. Performance will be affected differently, according to the usage pattern.
3. Usability can be heavily affected through the choice of authorization regime.
4. Operability is significantly affected. Most important is to determine what changes most frequently: subjects, objects or the authorization operations themselves.

Pattern 7: Bandwidth Throttling

Architectural solution: Prevention → Fairness

Problem. Facing Denial of Service attacks, a system might easily be overloaded with traffic, causing the system to stop responding to any new requests, both from the network or local input sources. This might even make it difficult to perform necessary corrective adjustments to the configuration of the system.

Solution. By imposing limits to the bandwidth certain services on a server can consume, remaining services are left operative and responsive [18]. Certain systems, such as Microsoft's Windows Server System 2003, allow the configuration of such throttles for individual services. This way, for example the web server on a server machine can be throttled (see http://www.microsoft.com/windowsserver2003).

Impact.
1. It is very hard to distinguish legitimate traffic from malicious traffic. Thus, bandwidth throttling will be a suboptimal solution to the problem. Some of the available bandwidth will remain unused. Overall performance during periods of normal operation will be lower.

2. Operability is positively affected. Once established, the throttle requires little, if any, attention.

Pattern 8: Biometric Authentication

Architectural solution: Prevention → Access control → Authentication → Context holder

Problem. It may be critical to use features of the subject to determine the subject's identity.

Solution. Biometric authentication is based upon using human features to distinguish between different users. It is ID-based, meaning that it bears upon the uniqueness of the person's features to assist in safe authentication. See also [42].

Impact.

1. Biometry is recognized to give very high levels of confidence in authentication. Certain biometric features, such as the human iris, are very difficult to counterfeit. Thus, in situations where high correct authentication is critical, biometrics might be a good choice. Biometrics give good support against repudiation.
2. Biometry can be expensive to deploy; it normally requires additional equipment to what is supplied as standard.
3. Biometric authentication can be resource intensive due to inherent tradeoffs between precision and processing resources required [36].
4. Biometry has a strong influence on usability aspects; although they might be both positive and negative, depending on the equipment used and the operative support for the equipment [55].

Pattern 9: Certificate

Architectural solution: Prevention → Access control → Authentication → Context holder

Problem. In asymmetric cryptography, there is a need to manage public keys. Not only must they be stored and transmitted, but also their validity must be ensured.

Solution. A certificate is a digitally signed data structure that contains information about an actor (a subject, person or application) and the actor's public key [47]. Certificates are issued by trusted organizations called certification authorities (CAs) after the CA has verified the identity of the subject. The CA is a kind of trusted third party (see Pattern 32: Trusted Third Party).

Impact.

1. There are only minimal effects on quality attributes from the use of certificates per se. They are merely relatively simple data structures. However, most organizations want a certain degree of trust in the management of them. Thus, e.g., certificate management issuing, expiration management and trust management through CAs, etc. is more involved and is likely to affect quality attributes more. See Pattern 32: Trusted Third Party).

Pattern 10: Clustering

Architectural solution: Recovery → Redundancy

Problem. Single points of failure within a given system architecture must occasionally be eliminated in order to obtain the wanted availability.

Solution. Clustering is an established pattern to avoid single points of failure. Essentially, it means implementing a virtual server farm, so that two or more servers appear as a unified server resource. Clustering may be implemented at different levels in the system architecture, for example by replicating the communication links, the servers, the disks, etc. However, care must be taken to ensure that the redundancy is not broken unintentionally. Clustering may be used for multiple purposes, such as load-balancing or failover [39]. Failover means that a surviving server may completely take over the tasks of a failed server. Some clusters can support both kinds of functionality.

Impact.

1. Clustering solutions for certain server systems may allow for load balancing between the servers during normal operation (i.e., no failures). This will improve performance.
2. Operability is affected negatively. Clustering systems can be hard to maintain, and they do require a good overview of the working of the system. For example, certain applications may not work correctly in a clustered environment due to the risk of losing consistency.
3. Maintainability may be positively affected. Certain clustering technologies allow single machines in the cluster to be "taken out" of the cluster for maintenance, such as operating system upgrades and hardware component replacement. The remaining machine(s) respond to service requests in the meantime.

Pattern 11: Cryptography

Architectural solution: Prevention → Obfuscation

Problem. Often, information must be transmitted over media that is open to other, potentially malicious actors. Confidentiality should still be maintained.

Solution. Cryptography address the problem of maintaining confidentiality of information by scrambling the information into unintelligible data using secret keys [17]. There are two main categories of cryptography (a) symmetric and (b) asymmetric. The former requires that both originator and recipient know the key. The latter is based upon two keys, a private and a public key. The private key should remain known only to the owner while the public key can be shared with anybody. However, it is important that the association between the private and the public key remains protected (see discussion on certificates in Pattern 9: Certificate).

Impact.

1. Performance is affected strongly by the choice of cryptographic solution. Generally, there are two rules (1) Asymmetric cryptography is more computationally intensive than the symmetric counterpart. (2) Accessing encrypted information by guessing the key is more difficult as the length of the key increases. However, longer keys also increase the computational resources required to encrypt and decrypt the information.
2. Operability is highly affected, mostly negatively, by the use of cryptographic techniques. The challenge of managing keys, i.e., distributing them, is complex. See also the discussion of trusted third party in Pattern 32: Trusted Third Party.

Pattern 12: Code Signing

Architectural solution: Detection → Embedded data integrity

Problem. In many systems, there is a need to install additional code after activation. Additional code, sometimes originating via the network, must be trustworthy.

Solution. A digital checksum, computed from a code unit such as a component, is appended to the code unit. The digital checksum is computed by the owner of the code unit using some predefined scheme known also by the recipient of the code unit. With a certain probability, modifications to the code unit will result in a mismatch between the code unit and the checksum. The recipient, upon detecting this mismatch may decide how to proceed (e.g., reject running the code, run the code with reduced privileges, etc.) If no mismatch is detected, the checksum provides evidence that the code unit was developed by a given organization and that it has not been modified after leaving the developing organization.

Impact.

1. Code signing increases the complexity of communication. There must be a regime for agreeing on the code in the checksum.
2. Performance. If the granularity of the code unit becomes very small, or the frequency of importing new code units is very high, the signing and verification of the code may consume significant computing resources on both machines.
3. Code signing, given appropriate schemes for generating the code, gives very good protection against integrity violations. They also support nonrepudiation.

Pattern 13: Cookie

Architectural solution: Prevention → Access control → Authentication → Context holder

Problem. A server may need to manage stateless client sessions. The client should only maintain a minimum amount of state.

Solution. The cookie is a data item that stores certain information about a client's session (typically with a web server). The cookie enables the server to associate a series of client requests with the same client without adding significant overheads to the data transmissions [32].

Impact.

1. Installability (i.e., ease of deployment) is improved. Cookies support the notion of thin clients (see also thin clients discussed in Pattern 30: Thin Client).
2. Resource behavior, in particular scalability, is improved. The cookie is small and only adds a small amount to the volume of information that needs to be communicated between client and server.
3. Security, and confidentiality in particular, can be damaged by the use of cookies as they are normally transmitted in clear text as part of the HTTP protocol. Although the protocol does provide a suggestive security mechanism [32], it does not include functionality to encrypt or otherwise protect the cookie. Alternative mechanisms for this purpose have been proposed in [44].

Pattern 14: Contextual Authorization

Architectural solution: Prevention → Access control → Authorization

Problem. Certain application domains benefit from dynamic authorization policies in which changing parameters about the actor should be used to control authorizations.

Solution. Contextual authorization is a sophisticated form of authorization scheme. Contextual authorization implies that it is knowledge about the actor's current context (or situation) that defines the appropriate authorizations [50, 56]. Many aspects of the user's context can be relevant for determining the appropriate authorizations; examples include the current role, the environment of the user or other attributes [16, 41].

Impact.

1. Suitability is improved. The authorizations can more precisely reflect the real needs of the application.
2. Operability for the end-user is improved, as the authorizations better reflect the actual need. However, from an administrative point of view, operability becomes more elaborate and difficult, as there are more parameters and configurations to manage. The rule-based approach described in [16] might be a viable approach to keep the efforts manageable.
3. Performance is negatively affected because a more complex set of rules related to the actor's context must be processed before access is granted or denied to a particular (set of) object(s). Additionally, context information must somehow be gathered and managed by the system.

Pattern 15: Digital Signatures

Architectural solution: Recovery → Liability transfer

Problem. The recipient of a document may want to establish trust in that the claimed originator sent it, and that the document has not been modified after leaving the claimed originator.

Solution. A digital signature is a kind of watermark on a piece of information. An actor can sign an electronic document to increase the trust of the recipient in believing that the document originated from the claimed actor and that it is not tampered with. Digital signatures require asymmetric cryptography and use public and private keys. The signer uses his private key to encrypt a hash value generated from the document. The recipient will, by decrypting the hash value with the signer's public key, verify that the same hash value is generated from the received document. It is worth noting that digital signatures per se do not provide absolute guarantees, but rather function as evidence of the authenticity of the signed document [43].

Impact.

1. Digital signature is primarily an instrument to avoid repudiation and will therefore assist accountability.
2. The use of digital signatures will improve integrity. Forgery can be detected using digital signatures.
3. Operability may be negatively affected. In large user communities the management of public keys requires a good infrastructure (PKI).

4. Performance is negatively affected. The use of asymmetric cryptography is resource intensive. Similarly, the public key of the originator must be obtained before the signature can be verified. This will add extra time to the verification process.

Pattern 16: IDS (Intrusion Detection System)

Architectural solution: Detection → Monitoring

Problem. Attacks via the network, e.g., the Internet, constitute a continuous threat to networked computers. Implementing protection against spurious attacks via the network should not be the concern of applications – it should be dealt with at the systems (or framework) layer. Another part of the concern is to how to determine the difference between normal activity and attack.

Solution. An IDS addresses the problem by implementing functionality for monitoring activity [47]. A good IDS provides a high accuracy of detecting attacks while maintaining low false alarm rates.

There are two principal types of IDS, categorized by the strategy they use for detecting attacks (a) anomaly detection and (b) signature detection. Orthogonal to these strategies, they can be implemented as network-based or host-based IDS. A network-based IDS is mainly concerned with scanning the network traffic, while a host-based IDS targets traces on the different host machines (inspecting, e.g., log files on the host machine). See for example [37].

Impact.

1. Security in general will benefit from IDS because if attacks are detected, they may be prevented.
2. Operability is negatively affected. IDS, in particular by anomaly detection types which may require frequent updating of configuration settings.
3. Performance. Depending on the level of sophistication of detection, the IDS will consume a certain amount of computing resources on the host while scanning logs (in case of host based IDS) or network resources (in the case of network based IDS) for the monitoring of network traffic.

Pattern 17: Layering

Architectural solution: Prevention → Compartmentalization

Problem. Assuming that some attacks are successful, we want to limit the consequence of the attack.

Solution. Each security barrier an attacker must conquer adds to the resources the attacker must posses before being successful [47, 54]. Layering prescribes that the software architecture consist of a set of layers (or parts) which communicate through a well-defined set of interfaces. Although the interfaces are well-defined, this does not mean that an attacker has easy access to them. Note: Tiering is a more specific kind of layering, in which the layers are deployed with separate machines. Layering is not concerned with the physical deployment of the parts.

Impact.

1. Layering may benefit integrity and confidentiality. An attacker who is able to modify or read data used in one layer need not obtain the same capabilities over data in another layer of the architecture.
2. Layering is an implementation of the tactic separation of concerns (not part of this reference architecture) and is a benefit for maintainability.
3. Layering may benefit deployment. Although layering is not directly concerned with deployment, layers normally form natural process boundaries. Process boundaries can again be used to select deployment configurations.

Pattern 18: Limited View

Architectural solution: Prevention → Access control

Problem. The user should not be allowed to perform operations that from a security perspective are known to cause access violations (or "access denied" responses). This will only create a less operable system.

Solution. Reducing the exposure of information to a minimum is beneficial for operability. It is also good for security in general and confidentiality in particular. Limited view implements a solution to this challenge by not exposing any information that is not explicitly authorized to the actor [59]. Information in this context also includes application programming interfaces (APIs. Thus, by not being visible, the actor is not presented with functions that are not allowed while the risk of breaches to confidentiality is reduced. The pattern can be implemented by for example views in an SQL database or by GUI code.

Impact.

1. Operability is normally very positively affected. By implementing the limited view pattern, there is no need to implement custom data hiding functionality. Additionally, a positive effect of limited view is that the probability of error messages due to security violations is reduced.
2. Complexity of implementation and maintenance can be high.

Pattern 19: Message Authentication Codes

Architectural solution: Detection → Embedded data integrity

Problem. During data communication, data might become corrupted. There are many reasons for this; one is a malicious threat agent that establishes attacks to hinder communication between two actors by attempting to influence the data stream.

Solution. The solution is similar to code signing. Data integrity verification checksums are added to the source data. They are computed by the sender from the original data using some predefined scheme known to both sender and recipient [47]. With a certain probability, modifications to the original data will result in a mismatch between the received data and the checksums. The recipient, upon detecting this mismatch, can request a retransmission or trigger an incident response.

Impact.

1. For security, error detection codes increase the likelihood of detecting integrity breaches.
2. Performance. Computing checksums and adding these checksums to the data stream requires resources and consumes a certain amount of the available bandwidth on the channel. Further, depending on the frequency of retransmissions, predictability of communication latency is reduced.

Pattern 20: Mirror Sites

Architectural solution: Recovery → Redundancy

Problem. A threat agent, such as natural disasters like fires and floods, may set a whole set of machines out of service.

Solution. Redundant processing facilities can withstand this kind of threat. The mirror sites pattern defines a backup site that can take over normal operation if the master site becomes nonoperational. The correct operation of such disaster recovery sites depends not only on technical measures, but also on good and well executed manual procedures performed by on-site personnel [1].

Impact.

1. The cost of establishing mirror sites is high. To reduce the impact of the extra cost, applications may be run on both sites during normal operation thus making better use of the available resources. Further, failover procedures must be tested frequently to guarantee correctness.
2. A high degree of availability can be achieved. For certain kinds of environment this justifies the extra cost.
3. Operability becomes more complex. Procedures for failover, i.e., the actual procedure for switching from one failed server to another functioning server, must be properly and frequently tested.

Pattern 21: Multi Barrier Security

Architectural solution: Prevention → Compartmentalization

Problem. Complex systems should not be completely jeopardized due to single incidents.

Solution. By constructing multiple barriers for potential threat agents, the probability that the whole system is damaged will be reduced. The solution can be implemented using for example operating system processes as barriers. Multi barrier security is also known as defense in depth [45].

Impact.

1. Multiple barriers imply more administration and thus decreased operability. The system can also become more cumbersome to use for end users.
2. Multiple barriers may help to separate concerns, thus improving maintainability.
3. Multiple barriers may reduce performance.

Pattern 22: Password

Architectural solution: Prevention → Access control → Authentication → Context holder

Problem. Representing evidence for proving identity.

Solution. The password is a bit string (normally in the form of a character string) given to the actor by which the actor is later authenticated. Upon request, the actor supplies the password to the system which subsequently compares it with the password stored in the password database for that particular actor. Supplying the correct password is evidence that the actor is the one claimed.

In addition to this simple scheme, security policies directing the use of passwords, may involve rules for how to construct legal passwords, how often they should be changed and so on.

Impact.

1. A good high-entropy password that is kept secret gives very high confidence authentication. However, passwords do not give any support against repudiation because they can be shared or stolen [42]. That is, passwords can only partly support accountability.
2. Operability is likely to be challenged. Remembering passwords requires a certain mental effort. Long passwords are required to give appropriate protection against password cracking threats. Some environments make entering passwords difficult, for example small computers without proper keyboards. Further more, users can only remember a finite number of different passwords. As security policies enforce changing passwords frequently, user's mental capabilities are stressed.
3. Complexity is low, thus bringing benefits to maintainability.

Pattern 23: Principle of Least Privilege

Architectural solution: Prevention → Compartmentalization

Problem. In large scale software systems the number of actors and assets can be very high. This complicates the task of ensuring the security of assets. A key challenge is to keep the cost of maintaining security at a low level.

Solution. The pattern (consistently referred to as a principle in literature, but prescriptive enough to be called a pattern) states that anything that is not expressively permitted is denied. Essentially, it creates compartments of information accessible by certain actors only [53]. In practice, it means that unless there is an authorization for an actor to access a subject, the authorization request is denied.

Impact.

1. Usability (in particular operability) is likely to be negatively affected by this pattern. It is inherently difficult to predict in advance the assets needed by an actor to complete a set of tasks [61].
2. Confidentiality and integrity benefit greatly. The risk of gaining unauthorized access to an asset is reduced.

Pattern 24: Reference Monitor

Architectural solution: Prevention → Single access point

Problem. Having to administrate multiple security related mechanisms decreases operability. It may also be a threat to security itself, as the risk of making errors increases.

Solution. The reference monitor is a single access point for a range of security related requests. There is no other way to get to the resource. It is comparable to an interpreter with added security checking functionality [53]. Thus, if the reference monitor is implemented correctly, the risk of error is reduced. It simplifies the design of the architecture. Today, the reference monitor pattern is often implemented by operating systems [28].

Impact.

1. The reference monitor may become a performance bottleneck as all security related requests must be processed by the monitor. The problem could be alleviated somewhat by inline reference monitors, but only at the expense of increased complexity [33].
2. Maintainability is improved through separation of concerns. The reference monitor is a single functional module that handles all security related requests.
3. The enforcement of integrity and confidentiality is easier to verify, and is thus probably improved, because the reference monitor is the only functional module dealing with security requests.

Pattern 25: Residual Information Protection

Architectural solution: Recovery → Fail-secure

Problem. Upon unexpected termination of programs, data from that program may remain in storage and be available through malevolent allocation of that storage by a threat agent.

Solution. The pattern residual information protection seeks to prevent leakage of information from one instantiation of a type of object to another instantiation of that type of object [28]. For example, the approach taken in MULTICS, by only clearing residues when the storage area was explicitly re-assigned [49], does not give the appropriate protection.

At the basic level, residual information protection requires encryption of stored data (see the discussion on Pattern 11: Cryptography). To further increase the security, mechanisms for automatically deleting information upon failed login attempts or after a timeout can be applied.

Impact.

1. On small devices, for which this pattern might be especially useful, there are limited computing resources. Encryption functionality might incur significant performance overheads.

Pattern 26: Resource Throttling

Architectural solution: Prevention → Fairness

Problem. Certain types of unwanted incidents are caused by threat agents generating excessive load on critical system resources, thus rendering the system unresponsive to operative or administrative requests (also known as Denial of Service attacks).

Solution. One solution to the problem is to limit the amount of resources that can be allocated by certain services, for example web servers. Although a negative consequence of this is that the maximum workload that the system can sustain will now be lower than necessary. An implementation of the pattern has been described in [40].

Impact.

1. Performance, during normal operation is lower. Some resources are reserved.

Pattern 27: Sandbox

Architectural solution: Prevention → Compartmentalization

Problem. It might be beneficial to incorporate code in an application that has been developed by organizations for which a suitable level of trust cannot be established.

Solution. This pattern involves building a contained execution environment for potentially hostile code (and thus resembles a "sandbox"). Attempts to perform actions that are not permitted by the security policy for the execution environment are prevented. Within this execution environment, any code can run without the risk of endangering the system [47]. Examples of this pattern are, e.g., the Java Virtual Machine, the Common Language Runtime from Microsoft, and efforts at Hewlett Packard that extend the operating system [60].

Impact.

1. The sandbox introduces another level of indirection (i.e., the interpreter or reference monitor that governs requests for system resources). This adds a certain performance overhead.
2. In terms of security, both confidentiality and integrity are greatly improved by the sandbox pattern.
3. To provide a flexible platform for component execution, the sandbox must include flexible security policy management features. There are significant differences in the approaches prescribed by Sun's Java Virtual Machine and the Common Language Runtime from Microsoft [46]. These must be considered when determining the technical platform for the application.

Pattern 28: Sentry

Architectural solution: Prevention → Controlled exposure

Problem. It is desirable to simplify the programming model for application programmers. Rather than having to understand all sorts of authorization mechanisms they should be provided with a simple mechanism that would allow them to operate upon data that was already authorized for the subject.

Solution. The sentry pattern was observed during an architecture evaluation session and has not been published in other literature. However, the sentry is similar to the proxy described in [47], but it is different in the sense that it essentially encapsulates the implementation of authorization rules. The sentry is a kind of proxy for designated classes. It provides a security-amended interface that is identical to the underlying class. Use of the sentry is not enforced however, the programmers also have access to the underlying class, without the authorization checks built in.

Impact.

1. The sentry is a kind of proxy pattern, thus it does introduce extra overhead in processing by adding another level of indirection and most likely extra processing required for the authorization procedures. Having said that, the sentry also gives a very good potential for optimizing the determination of authorizations. Because the sentry instance is associated to only one object in the program, security parameters related to that object can be cached in the sentry, thus giving gains in terms of performance.
2. The sentry simplifies the programming model for the application programmer. Thus maintainability is improved.

Pattern 29: Single Sign On

Architectural solution: Prevention → Access control → Authentication

Problem. A very common problem in distributed systems is the burden felt by the users to remember a distinct password for any single system they want to use. Therefore, many users choose shorter passwords than recommended or even decide to write down passwords on paper notes.

Solution. The solution offered by this pattern is to integrate the authentication subsystems of multiple systems into a single, coherent whole [47]. Thus, by completing authentication via the single sign on solution, the actor is authenticated on all subsystems.

Impact.

1. The same security benefits as for Pattern 3: Authentication.
2. Reliability: The single sign on module constitutes a single point of failure. It therefore constitutes an attractive target for attacks such as Denial of Service attacks.
3. Performance: The single sign on module is responsible for all authentication requests for the subsystems. If not scaled appropriately, it may become a bottleneck.
4. Operability is increased: All administration can be performed through the same system.
5. Usability: Users will observe increased usability as they now need only a single authentication procedure.
6. Replaceability is hampered: Single sign on solutions typically require a certain amount of adaptation to the applications taking part.

Pattern 30: Thin Client

Architectural solution: Prevention → Compartmentalization

Problem. Certain environments are particularly vulnerable to attacks. Computers used in such environments should operate upon the least possible amount of assets, and seek to reduce the storage of assets locally.

Solution. The thin client pattern is a variant of the client server pattern [19]. One aspect of this pattern is that there is a physical boundary between the server and the client. In addition, the thin client pattern prescribes that only the minimal amount of processing and storage is done at the client (in contrast to thick- or self-reliant clients). Although there is a continuum between thin and thick clients, an example of a thin client would be a web-browser.

Impact.

1. Less information and code is stored on the client. This reduces the consequences if a threat agent is able to compromise the machine. Thus, integrity and confidentiality will be improved.
2. Time behavior is likely to be somewhat impaired. It may be difficult to achieve good responsiveness in a GUI which is heavily bound by the latency of the underlying network.
3. Installability is greatly improved by the thin client pattern. Clients are small, and require very little technical infrastructure on the client machine.

Pattern 31: Timeout

Architectural solution: Recovery → Fail-secure

Problem. In case a session is left open for an extended period of time, there is a chance that the actor forgot to close it. This creates a potential for unwanted incidents.

Solution. The timeout pattern automatically locks the session after a certain period of inactivity on a session [20]. The exact duration is determined by a timeout value, used in implementations of the pattern. The actor will have to re-establish the session if the lock has been triggered.

In practice, multiple variants of the timeout can be implemented. At the GUI level, a common approach is to lock the keyboard/screen after a period of inactivity. The user will then have to unlock the GUI in order to continue working. Typically, this is unlocked using a certain password. Another variant of the system lock is found at the communication session level. Upon establishing communication sessions, a timeout starts running. If the session is unused for the specified timeout period, the session will close and it will have to be re-established again.

Impact.

1. Operability is somewhat reduced. Depending on the timeout value of the timeout pattern, the actor can be forced to perform a number of unnecessary attempts to re-establish a session.
2. Complexity is negatively affected. Any actor interacting with a system that implements timeouts will have to consider the situation that a session expires and requires renewal.

Pattern 32: Trusted Third Party

Architectural solution: Prevention → Service provider

Problem. For transactions between two or more actors, a certain level of trust must be present in order to facilitate efficiency.

Solution. A solution to providing trust is a trusted third party (TTP). The TTP is a security authority or an associated agent that is trusted by the participating actors [47]. A TTP can support multiple services in a security architecture, for example authentication requests, authorization requests or issuing/revocation of certificates with public keys for digital signatures or code signing.

The TTP is a network entity which responds to requests from the trusting actors.

Impact.

1. The availability of the TTP is critically important for the actors. Thus, the reliability of both the network connectivity and the machine upon which it runs will have direct effects for the working of the cooperating actors.

8.7 Using the Reference Architecture

The reference architecture provides decision support for architecture derivation and evaluation in a product line context. It is intended to function as a tool for this purpose. Below we provide some guidance for its use.

8.7.1 Architecture Derivation

Each application will have specific quality requirements, these requirements must be accommodated in an application dependent quality model. The application dependent quality model is obtained by determining which quality attributes of the generic quality model

are relevant, and then resolving the appropriate variation points in the scenarios using the application's specific quality requirements.

Using the variants suggested by the scenario, one should consider the architectural solutions referred to. Multiple solutions may help to satisfy the requirement; in that case one should select the solution (or solutions) that have the best total effect on the quality attributes. Priorities of the scenarios, determined for each product, may help to resolve such trade-off situations.

The architectural solutions selected here, together with any prescribed architectural solutions for the product line architecture, will constitute the architectural solutions preferred for the application architecture.

8.7.2 Architecture Evaluation

The requirements that an architecture should fulfill will vary over time. Thus, it makes sense to evaluate the architecture for potential gaps between the expectations and the quality attributes supported by the architecture.

The reference architecture is a tool that can support this process. Similar to the process for architecture derivation, the quality requirements that the architecture should fulfill must be determined first. By examining the quality model and the decision model embodied by this reference architecture, the scenarios that best represent the requirements can be selected and their variation points resolved. Then, the architectural solutions used in the actual architecture can be compared with the ones suggested by the scenarios. Divergences can then be used as a foundation for subsequent analyses of potential architectural changes.

8.7.3 Evolution of the Reference Architecture

Capturing valuable architectural design knowledge is an overall goal of the reference architecture. This is not a static activity. As new knowledge is collected in the organization it should be reflected in the reference architecture. The reference model accommodates this in several ways.

As previously noted, the decision model is based upon quality requirements formulated as scenarios, extracted, refined and generalized from business requirements relevant for the four companies contributing to the decision model (see Sect. 8.5). This approach has given us a useful starting point for the decision model, but the model should be maintained in order to provide maximum value to the organization using it. A significant advantage of the proposed decision model is its extensibility. New scenarios can be added, existing ones can be improved. Thus, the adopting company may tailor the model to internal knowledge and experience. An example of this is the adding of new responses to existing scenarios in order to increase variability in the application architecture.

The security architecture language has been developed by reviewing a large amount of security related sources. No claims can be made about its completeness or correctness however. Instead, it is a proposal that we have found useful in our work together with companies. As adopting companies gain knowledge of their own language, the proposed language can be extended, modified and maintained. For example, it is likely that compa-

nies use specializations of the described security architectural solutions. In that case, it will make sense to include also these in a revised language.

Hopefully, these efforts will help to ensure that the reference architecture remains an effective tool for the organization in its efforts to build better architectures. Similarly, it supports the management of this architectural knowledge within the organization.

8.8 Validation

There were two main questions we wanted to investigate through the development of this reference architecture: Firstly, is it viable to represent architectural security knowledge in a reference architecture? Secondly, and most important: Does the approach give value to a software company that wants to develop software architectures concerned with security?

By constructing such a reference architecture and subsequently describing it, we believe we have affirmed the first question. It has also been presented and discussed several times at various project meetings and the comments that were received have been used to revise and improve it.

To answer the second question we have used the reference architecture to guide architectural design and support architecture evaluation in three different software product companies. These three were also contributing to the construction of the reference architecture. The fourth company that contributed to the reference architecture did not participate in its validation. For reasons of confidentiality we denote them as A, B and C. Company A is a large international telecommunications solutions provider, companies B and C are medium sized software houses in the information systems domains. We did not have the resources available to do complete security architectural designs or reviews in any of the companies. Accepting this, we chose to focus on certain aspects of the various application architectures and selected a few particularly important capabilities in each. Here we cannot reveal all details of the security reviews but we'll provide examples from each of them.

Generally, for all the companies we observed a very positive attitude towards a systematic review of security architecture. During validation in company A we did not have the time to consider underlying threat and risk assessments, but for certain key assets we could do this in companies B and C. Although this should be done thoroughly, we learned from both B and C that this produced very valuable incitements for future development plans. Particularly the phase of determining what value an asset actually represented to the range of stakeholders generated intense discussions among the participants. Our belief is that even such rudimentary reasoning on this topic is forgotten in many companies.

8.8.1 The Quality Model

Our quality model, organizing security qualities into gradually more specialized quality attributes, proved to be a significant help in reviewing and specifying requirements. The main problem that the quality model addressed was confusion in terminology. As the hype around security has flourished it appears that the precision in what is really meant has been lost on the way. Many of the people we talked with confused qualities with technical

solutions. As an example, authentication was often denoted as a quality attribute for a system. This is wrong. As we have explained, authentication is simply a means to achieve security qualities – primarily accountability. We believe that the quality model improves the communication among the stakeholders and increases the awareness of the wide range of security qualities that may be important for a software system. We also believe that it stimulates companies to perform more thorough threat assessments in order to further improve the security requirements engineering process.

While working with A we had to cope with a certain disagreement in terminology. It took some time before our model was accepted. However, we think it is crucial to take this seriously as a common vocabulary is critical for the work.

B worked with us over a longer period of time and we had a deeper cooperation with them. In fact, B provided significant input to our work in constructing the quality model. The input came through a range of inherent security concerns related to the release of a software platform for a new generation of web-based products. The quality model was reviewed and improved while working with them.

For C we considered security aspects related to a capability in their product to use e-mail as a communications channel to distribute highly sensitive business information to selected partner companies. Referring to the risk assessment we had done and going through the various elements of the quality model it was easy to conclude that confidentiality and accountability were the prime concerns. Integrity and availability was less important as the information exchange was one-way and there were minimal impacts stemming from moderate delays in the distribution.

8.8.2 The Decision Model

As expected, the decision model proved to benefit from contributions of business requirements originating in companies within different application domains. The existing scenarios were generic enough to cover the requirements in the companies, but one company felt that increased value would be obtained with more scenarios directly targeting technological infrastructure. To accommodate such needs, from this company and others, the decision model was designed to support evolution in the scenarios. New scenarios may be added in order to better capture the needs of particular organizations. It should be noted here that we had to concentrate our efforts at each of the companies. Thus, only the most highly prioritized scenario in each company was reviewed due to time and resource constraints.

Company A was particularly concerned about integrity aspects related to the use of third party components in large-scale distributed systems. Scenario I.1 was the primary reasoning framework for evaluating different architectural design alternatives. The different variants helped the company to determine the most appropriate architectural design, and through discussions of the impacts of the relevant architectural solutions the favored alternative was variant 3.

In B we also worked primarily with integrity. An important concern was to provide effective, yet easy to use abstractions for programmers that would assist in maintaining protection against unwanted modification of data. Scenario I.3 provided the reasoning framework for the architectural decisions. Further, B had concerns related to the potential exposure of corporate data while users were in a mobile context. We found that scenario

I.5 gave good architectural guidance in this case. Variants 3 and 4 were both relevant for further improvements of the architecture.

For C, the reference architecture did not provide applicable scenarios dealing with the particular problem of distributing information via e-mail. However, from the review of the architecture language, we had already selected a candidate architectural solution that we used to create a more detailed technical design.

8.8.3 The Security Architecture Language

The security architecture language was a good help for participating software architects that were searching for appropriate solutions in the security architectural design space. Of particular benefit was the clear structure and classification of solutions into detection, prevention and recovery. This triggered an exploratory review of the different solutions, which again were used to increase the accuracy in the resolved variation points in the scenarios.

In the case of A we concluded that the need to provide high performance and a minimum of added complexity in the administration of the solution motivated the use of tactics similar to the recommendation of variant 3 of scenario I.1, which is Pattern 12: Code Signing. In addition A wanted to explore open source components. In the platform, which is currently being implemented, open source components are investigated. It is not unreasonable however to compare the open source community with a code signing party.

Company B found, in accordance with the scenarios mentioned above, that a combination of thin clients and contextual authorization provided the most significant benefits in terms of integrity. A role-based security authorization scheme is currently under development for the new platform.

For C we determined that prevention was the most beneficial tactic. Since we were dealing with sensitive information, and the fact that even knowing that an actor is sending information to another named actor could be misused, the recovery tactic was abandoned. The detection tactic was also abandoned because we did not consider malicious attacks as the threat. Prevention tactics were investigated and the service provider solution was selected as the most promising candidate. We decided to examine the directory pattern in our further design (only illustrated in the reference architecture, see [47] for more information on the pattern). The idea behind using the service provider pattern was to make the selection process of e-mail addresses more secure, yet user friendly.

8.8.4 Summary

In summary, the reference architecture proved to be a good starting point for security architecture design. Naturally, as the reference architecture is a representation of knowledge at an abstract level it only provides general architectural design guidance and specialist domain knowledge must be used in the later design phases of actual architectures.

8.9 Related Work

Security has seen a formidable growth in interest during the last few years. A number of journals, conferences and other publications have been established to foster research in the field. Similarly in industry, security has been established as a key concern for those responsible for managing, buying or developing ICT systems. It appears reasonable to infer that with society's increasing dependence on ICT systems, the very same systems have become lucrative targets for persons with intentions to gain benefits through malevolent actions.

A key pillar of this work is the established relationship between quality attributes and architectural solutions. Work done at SEI on ABASs [31] and later within the ADD method [3] shows that quality properties can be associated with architectural tactics and patterns.

Risk assessments to the appropriate level of detail should be the foundation for any work related to the design of evaluation systems having to cope with security requirements. In [11], Cavusoglu et al. describe an evaluation framework that can be used in order to determine risks and also support the selection of appropriate security controls (artifacts related to countermeasures). They also bring up the need for architectural decision support in order to build architectures that provide the required level of security. The proposed reference architecture goes further into the architectural design aspects, by providing a more elaborate decision support framework – both in the area of security architecture language and in terms of a richer quality model. SAEM is a more pragmatic approach to the proposed reference architecture, focusing more on concrete risks and technologies than quality modeling and architectural design [9].

All professional activities related to security must be considered in terms of economical viability. The reference architecture presented here does not address this issue, but the reader should consult sources such as [24]. Consideration to economical justifications should precede the architectural design phase.

High level security tactics have been discussed by, e.g., Romanosky [48] although without any structured approach to their use or applicability. Such efforts are nevertheless useful as documentation of architectural solutions and effects. Further, a large amount of security design knowledge has been collected and made available through work in the security pattern community [5, 45, 48, 52, 54, 59]. This contribution does not attempt to repeat these efforts, but uses findings from that community in order to build increased credibility in the association of effects and architectural solutions. The language presented here makes numerous references to work from the pattern community. Further, by reviewing many of these efforts we have been able to determine important relationships between the architectural solutions.

Chapter 9 presents a framework for architectural evolution based on architectural recovery as a means to ensure alignment with nonfunctional requirements. The assumption is that these requirements are not properly addressed in currently available tools. While the reference architecture presented here is a conceptual model for quality driven architectural design, it does not go into the same level of detail with respect to the technologies. Neither does it address the problem of distributed service management.

8.10 Conclusions and Future Research

This chapter presented a reference architecture for security in product lines. It has enabled us to show that it is feasible to provide useful decision support, based upon architectural design knowledge, for companies developing product lines in which security is a quality requirement.

Software architectural design does still have similarities with an art form. And there is a significant amount of manual labor involved in making the appropriate tradeoffs of the design alternatives and subsequently determining the final detailed design. However, we believe that the proposed reference architecture can support those who need to embark on such tasks.

Reference architectures play two roles. One is to generalize and provide abstractions useful to a wide range of systems. The second role is to be a platform from which specific architectures can be instantiated. The presented reference architecture has been developed with generality as its primary objective. Our intention is that companies wanting to use it should adapt it to the specific needs of the product line in development. As part of our future research we will investigate the potential of specialization of the reference architecture for a specific product line.

Acknowledgments

The work presented here is the result of collaboration with – and kind assistance from – a number of people in the ITEA project FAMILIES and its Norwegian co-project FAMILIER (the latter sponsored by the Norwegian Research Council and led by ICT-Norway). Some deserve particular mention; Ivar Sandstad, Jens Glattetre from SuperOffice ASA, Frank Mikalsen from Finale, Miguel Ángel Oltra Rodríguez from Telvent, Frode Nergård from EDB Telesciences, Juha Savolainen from Nokia, Timo Käkölä from University of Jyväskylä, Eila Niemelä from VTT, and last, but not least, Juan Carlos Dueñas from The Technical University of Madrid provided valuable comments that helped shape this work. We would also like to thank our colleague Odd Nordland in SINTEF ICT for improving the language of the chapter.

References

1. Allen J, Gabbard D, and May C (2003) Outsourcing Managed Security Services In: Security improvement module, CMU/SEI-SIM-012. 2003, Carnegie Mellon, Software Engineering Institute: Pittsburg, PA.
2. Bachmann F, Bass L, and Klein M (2003) Deriving Architectural Tactics: A Step toward Methodical Architectural Design Technical report, CMU/SEI-2003-TR-004. 2003
3. Bass L, Clement P, and Klein M (2003) Software Architecture in Practice. 2 ed. Addison Wesley.
4. Bayer J (2003) Design for Quality. In: Linden FVd (ed) 5th Int'l Workshop on Product Family Engineering. Springer, Berlin Heidelberg New York
5. Blakley B and Heath C (2004) Security Design Patterns. The Open Group.
6. Bollinger T, Voas J, and Boasson M, *Persistent Software Attributes.* IEEE Software, 2004. **21**(6): p. 16-18.

7. Bosch J (2000) Design and Use of Software Architectures - Adopting and Evolving a Product-Line Approach. Addison-Wesley.
8. Bosch J and Molin P (1999) Software Architecture Design: Evaluation and Transformation. In: IEEE Conference and Workshop on Engineering of Computer-Based Systems. IEEE Computer Society Press. p. 4-10.
9. Butler SA (2002) Security Attribute Evaluation Method: A Cost-Benefit Approach. In: International conference on software engineering. ACM Press. p. 232-240.
10. Bühne S, Chastek G, Käkölä T, Knauber P, Northrop L, and Thiel S (2003) Exploring the Context of Product Line Adoption. In: Linden FVd (ed) 5th International workshop on Product Family Engineering. Springer, Berlin Heidelberg New York. p. 19-31.
11. Cavusoglu H, Mishra B, and Raghunathan S, *A Model for Evaluating It Security Investments.* Communications of the ACM, 2004. **47**(7): p. 87-92.
12. Chang S, Chen Q, and Hsu M (2003) Managing Security Policy in a Large Distributed Web Services Environment. In: 27th Annual International Computer Software and Applications Conference (COMPSAC'03). IEEE Computer Society Press. p. 610-621.
13. Chung L, Nixon BA, Yu E, and Mylopoulos J, eds. Non-Functional Requirements in Software Engineering (2000). The Kluwer International Series in Software Engineering, Basili V (ed). Kluwer Academic Publishers. 439pp.
14. Clements P, Kazman R, and Klein M (2002) Evaluating Software Architectures: Methods and Case Studies. Addison Wesley.
15. Clements P and Northrop L (2002) Software Product Lines: Practices and Patterns. The Sei Series in Software Engineering Addison Wesley.
16. Covingtony MJ, Fogla P, Zhan Z, and Ahamad M (2002) A Context-Aware Security Architecture for Emerging Applications. In: 18th Annual Computer Security Applications Conference (ACSAC'02). IEEE Computer Society. p. 249-258.
17. Denning DE and Denning PJ, *Data Security.* ACM Computing Surveys, 1979. **11**(3). p. 227-249.
18. Douligeris C and Mitrokotsa A, *Ddos Attacks and Defense Mechanisms: Classification and State-of-the-Art.* Computer Networks, 2004. **44**(5): p. 643-666.
19. Duchessi P and Chengalur-Smith I, *Client/Server Benefits, Problems, Best Practices.* Communications of the ACM, 1998. **41**(5): p. 87-94.
20. Eguiluz HR and Barbacci MR (2003) Interactions among Techniques Addressing Quality Attributes In: Technical report, CMU/SEI-2003-TR-003. 2003.
21. Fernandez EB and Pan R (2001) A Pattern Language for Security Models. In: PLoP 2001.
22. Firesmith DG (2003) Common Concepts Underlying Safety, Security, and Survivability Engineering Technical note, CMU/SEI-2003-TN-033. 2003, Software Engineering Institute, Carnegie Mellon University
23. Gates C and Slonim J (2003) Owner-Controlled Information. In: New security paradigms workshop. ACM Press. p. 103-111.
24. Gordon LA and Loeb MP, *The Economics of Information Security Investment.* ACM Transactions on information and system security, 2002. **5**(4): p. 438-457.
25. Hallsteinsen S, Fægri TE, and Syrstad M (2003) Patterns in Product Family Architecture Design. In: Linden FVd (ed) PFE-5. Springer Verlag. p. 261-268.
26. Hallsteinsen S, Fægri TE, and Syrstad M (2003) Patterns in Product Family Architecture Design. In: Linden FVd (ed) 5th International workshop on Product Family Engineering. Springer, Berlin Heidelberg New York. p. 261-268.
27. IEEE (2000) Ieee Standard No. 1471-2000: Recommended Practice for Architectural Description of Software-Intensive Systems. IEEE: http://shop.ieee.org/store/.
28. ISO/IEC (1999) 15408 Common Criteria for Information Technology Security Evaluation. v2.0 ed. Nat'l Inst. Standards and Technology Washington, DC.
29. ISO/IEC (1991) Fcd 9126-1.2: Information Technology - Software Product Quality. Part 1: Quality Model.
30. Jung H-W, Kim S-G, and Chung C-S, *Measuring Software Product Quality: A Survey of Iso/Iec 9126.* IEEE Software, 2004. **21**(5): p. 88-92.
31. Klein M and Kazman R (1999) Attribute-Based Architectural Styles. In: SEI Technical Report, CMU/SEI-99-TR-022. 1999.
32. Kristol D and Montulli L (2000) Http State Management Mechanism (Rfc2965). http://www.watersprings.org/pub/rfc/rfc2965.txt.

33. Landwehr CE, *Computer Security.* International Journal of Information Security, 2001. **1**(1): p. 3-13.
34. Linden Fvd, *Software Product Families in Europe: The Esaps and Café Projects.* IEEE Software, 2002. **19**(4): p. 41-49.
35. Lindqvist U and Jonsson E, *A Map of Security Risks Associated with Using Cots*, in *IEEE Computer*. 1998. p. 60-66.
36. Matyás V and Ríha Z, *Towards Reliable User Authentication through Biometrics.* IEEE Security & Privacy, 2003. **1**(3): p. 45-49.
37. McHugh J, *Intrusion and Intrusion Detection.* International Journal of Information Security, 2001. **1**(1). p. 14-35.
38. Microsoft (2002) Building Secure Asp.Net Applications. Microsoft: www.microsoft.com.
39. Microsoft (2003) Enterprise Solution Patterns Using Microsoft .Net. Microsoft Corp.: http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dnpatterns/html/Esp.asp.
40. Min BJ, Kim SK, and Choi J-S (2003) Secure System Architecture Based on Dynamic Resource Reallocation. In: Chae K and Yung M (eds) Information Security Applications, 4th International Workshop, WISA 2003. Springer. p. 174-187.
41. Motta GHMB and Furuie SS, *A Contextual Role-Based Access Control Authorization Model for Electronic Patient Record.* IEEE Transactions on information technology in biomedicine, 2003. **7**(3): p. 202-207.
42. O'Gorman L, *Comparing Passwords, Tokens, and Biometrics for User Authentication.* Proceedings of the IEEE, 2003. **91**(12).
43. Oppliger R and Rytz R, *Digtal Evidence: Dream and Reality.* IEEE Security & Privacy, 2003: p. 44-48.
44. Park JS, Sandhu R, and Ahn G-J, *Role-Based Access Control on the Web.* ACM Transactions on information and system security, 2001. **4**(1): p. 37-71.
45. Peteanu R (2001) Best Practices for Secure Development. http://www.mkaz.com/ref/secure_webdev-3.0.pdf.
46. Probst S, Essmayr W, and Weippl E (2002) Reusable Components for Developing Security-Aware Applications. In: 18th Annual computer security applications conference (ACSAC'02). IEEE. p. 239-248.
47. Ramachandran J (2002) Designing Security Architecture Solutions. Wiley.
48. Romanosky S (2002) Enterprise Security Patterns. In: 7th European conference on pattern languages of programs (EuroPLoP). http://hillside.net/patterns/EuroPLoP2002/
49. Saltzer JH, *Protection and the Control of Information Sharing in Multics.* Communications of the ACM, 1974. **17**(7): p. 388-402.
50. Sandhu RS and Samarati P, *Access Control: Principle and Practice.* IEEE Communications Magazine, 1994. **32**(9): p. 40-48.
51. Schmidt DC and Buschmann F (2003) Patterns, Frameworks, and Middleware: Their Synergistic Relationships. In: International conference on software engineering. IEEE Computer Society. p. 694-704.
52. Schneider EA (1999) Security Architecture-Based System Design. In: New security paradigms workshop. ACM Press. p. 25-31.
53. Schneider FB, *Least Privilege and More.* IEEE Security & privacy, 2003: p. 55-59.
54. Shumacher M, ed. Security Engineering with Patterns: Origins, Theoretical Model, and New Applications (2003). Lecture Notes in Computer Science. Vol. 2754. Springer Verlag pp.
55. Smith SW, *Humans in the Loop: Human-Computer Interaction and Security.* IEEE Security & privacy, 2003. **1**(3): p. 75-79.
56. Ting TC (1993) Modeling Security Requirements for Applications. In: Conference on Object Oriented Programming Systems Languages and Applications. ACM Press. p. 305.
57. Viega J and Messier M, *Security Is Harder Than You Think*, in *ACM Queue*. 2004. p. 60-65.
58. Yee K-P (2002) User Interaction Design for Secure Systems. In: Fourth International Conference on Information and Communications Security. Springer Verlag. p. 278-290.
59. Yoder J and Barcalow J (1998) Architectural Patterns for Enabling Application Security. In: In Proceedings of the 4th Conference on Patterns Language of Programming (PLoP'97). p. 1-31.
60. Zhong Q and Edwards N, *Security Control for Cots Components.* IEEE computer, 1998. **31**(6): p. 67-73.
61. Zurko ME and Simon RT (1996) User-Centered Security. In: New Security Paradigms Workshop. ACM Press. p. 27-33.
62. Aagedal JØ, Braber Fd, Dimitrakos T, Gran BA, Raptis D, and Stølen K (2002) Model-Based Risk Assessment to Improve Enterprise Security. In: Sixth International Enterprise Distributed Object Computing Conference (EDOC'02). p. 51-62.